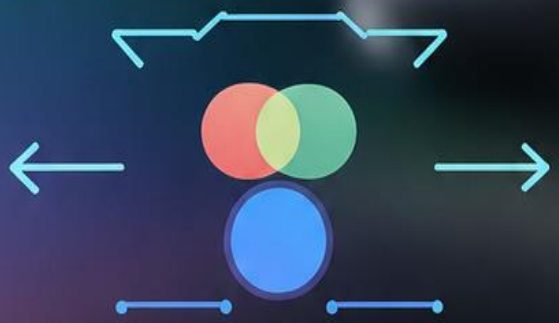
AI-Powered Intelligent Vehicle Pricing System

CatBoost Regressor | R2 = 0.94

Objective



Predict used car prices using machine learning and vehicle features

Tech Stack

React FastAPI Python

Methodology

Data

↓

Feature Engineering

↓

CATBOOST

↓

Prediction

Results

MAE: 167,520

R2: 0.94

AI-Powered Intelligent Vehicle Pricing System

Prepared By:

ID	Name	Department
352250721	Amir Mansour Eid Mohammed	Statistics & CS
352250964	Muhammed Amr Muhammed	Statistics & CS
352250744	Hazem Amr Mohammed	Statistics & CS
352250742	Hazem Elsayed Abou Elwafaa	Math & CS
352250778	Zeyad Sayed Mohammed	Math & CS
352250797	Seif Hatem Abdelhaleem	Math & CS

Supervised By:

Dr. Ahmed Madboly

- ## Team Grades

ID	Name	Department	Grade			
			Discussion	Follow-Up	Presentation	Total
352250721	Amir Mansour Eid Mohammed	Statistics & CS				
352250964	Muhammed Amr Muhammed	Statistics & CS				
352250744	Hazem Amr Mohammed	Statistics & CS				
352250742	Hazem Elsayed Abou Elwafaa	Math & CS				
352250778	Zeyad Sayed Mohammed	Math & CS				
352250797	Seif Hatem Abdelhaleem	Math & CS				

- ## JUDGMENT COMMITE

Name	Signature

Table of Contents for Chapters

Project Vision

The vision of this project is to develop an AI-powered intelligent car price prediction system capable of accurately estimating vehicle market prices using advanced machine learning and ensemble learning techniques. The system is designed to analyze real-world automotive market data and identify the relationship between vehicle specifications and pricing behavior in order to generate reliable and data-driven price predictions.

This project aims to combine the fields of Artificial Intelligence, Data Science, Business Analysis, and Software Engineering to create a complete intelligent system rather than only developing machine learning models. The system focuses on transforming raw automotive market data into meaningful insights that support decision-making and improve pricing accuracy in the automotive industry.

The project utilizes multiple machine learning regression algorithms, including ensemble and boosting techniques, to compare performance and determine the most effective model for vehicle price prediction. In addition, feature engineering and data preprocessing techniques are applied to enhance model performance and improve prediction accuracy.

The proposed system is intended to provide a practical and user-friendly solution that can assist:

- Customers in estimating vehicle prices
- Car sellers in determining competitive market values
- Automotive dealerships in pricing optimization
- Businesses in analyzing automotive market trends

The project also focuses on building a professional AI-based system that reflects the structure and workflow of real-world intelligent pricing systems used in modern industries. Therefore, the project includes several important components such as:

- Data preprocessing
- Exploratory data analysis
- Feature engineering
- Machine learning model development
- System analysis
- System design

- Model evaluation
- Web-based system implementation

Chapter 1 : Project Introduction and Importance

1.1 Introduction

The automotive market has experienced significant growth over the last decade, resulting in a rapid increase in the number of used vehicles available for sale. Determining the appropriate price of a vehicle has become a challenging task because the value of a car depends on multiple factors, including brand, model, mileage, fuel type, transmission type, engine specifications, age of the vehicle, and other characteristics. In many cases, buyers and sellers rely on personal experience or subjective judgments, which often leads to inaccurate pricing and unfair transactions.

Incorrect vehicle valuation can negatively affect both individuals and businesses. Buyers may pay prices higher than the actual market value, while sellers may undervalue their vehicles and incur financial losses. Consequently, there is a growing need for intelligent systems that can estimate vehicle prices accurately and consistently using data-driven approaches.

1.2 Problem Statement

Estimating the price of used vehicles is a complex problem due to the large number of variables that influence market value and the nonlinear relationships among these variables. Traditional pricing methods depend heavily on human expertise and market observations, making them time-consuming and prone to inconsistencies.

The absence of reliable and automated pricing mechanisms creates challenges for customers, dealerships, and online vehicle marketplaces. Therefore, developing an intelligent prediction system capable of analyzing vehicle characteristics and producing accurate price estimations has become an important research problem.

1.3 Background and Research Foundation

The problem of car price prediction has attracted considerable attention from researchers in recent years. Several studies have demonstrated that machine learning techniques can provide more accurate estimations compared with conventional approaches.

The first research paper, entitled **"Used Car Price Prediction Using Machine Learning Techniques"**, investigated the application of different regression algorithms to estimate vehicle prices. The study demonstrated that ensemble learning models achieve better

performance because they are capable of capturing complex relationships between vehicle characteristics and price.

The second research paper, entitled **"Predicting Car Pricing in Germany Using Different Machine Learning Algorithms with AutoScout24 Dataset"**, compared multiple machine learning models using real-world automobile data. The study highlighted the effectiveness of advanced boosting algorithms in improving prediction accuracy and emphasized the importance of feature engineering and hyperparameter optimization.

Inspired by these studies, the present project adopts a similar machine learning framework and aims to further evaluate and compare advanced ensemble methods using a dataset containing more than 200,000 vehicle records. Special attention is given to data preprocessing, feature engineering, and model optimization to achieve higher predictive performance.

1.4 Motivation

With the increasing digitization of automotive marketplaces, there is a growing demand for intelligent systems that provide fair and reliable vehicle price estimations. Machine learning techniques offer an opportunity to automate this process and reduce the dependency on subjective human judgment.

This project was motivated by the need to build a practical and accurate prediction system that can assist buyers, sellers, dealerships, and online platforms in making better pricing decisions.

1.5 Project Objectives

The main objective of this project is to develop an intelligent car price prediction system based on machine learning algorithms. Specifically, the project aims to:

- Analyze and understand the characteristics of vehicle data.
- Perform comprehensive data preprocessing and feature engineering.
- Compare the performance of several machine learning models.
- Optimize model hyperparameters using RandomizedSearchCV.
- Identify the most effective algorithm for price prediction.
- Build a reliable AI-based system capable of generating accurate price estimations.
- Provide practical insights that can support decision-making in the automotive market.

1.6 Research Papers Related to the Problem

Several published studies have investigated the problem of vehicle price prediction. Among them, two papers played a major role in shaping the methodology adopted .

Research Paper 1

Advanced Feature Engineering and Machine Learning Techniques for High Accurate Price Prediction of Heterogeneous Pre-Owned Cars

This paper focuses on improving used car price prediction by applying advanced feature engineering and machine learning techniques. The research explains how preprocessing and transforming raw vehicle data can significantly improve the performance of predictive models.

One of the most important ideas discussed in this paper is that vehicle pricing is affected by many connected factors such as mileage, fuel type, vehicle age, transmission type, and engine performance. Because of this complexity, traditional prediction methods are often not accurate enough, while ensemble learning and boosting algorithms can better capture these hidden relationships.

The study compares several machine learning models and demonstrates that boosting algorithms such as CatBoost, LightGBM, and XGBoost achieve stronger predictive performance compared to traditional regression methods.

This paper had a direct impact on our project, especially in:

- Applying feature engineering techniques
- Improving data preprocessing
- Using ensemble learning models
- Comparing multiple regression algorithms
- Applying hyperparameter tuning methods
- Evaluating models using performance metrics

The paper also inspired the idea of creating engineered features that help the models better understand vehicle usage and market behavior. As a result, feature engineering became an important part of our system development process.

Research Paper Link

Advanced Feature Engineering and Machine Learning Techniques for High Accurate Price Prediction of Heterogeneous Pre-Own Cars

Research Paper 2

Predicting Car Pricing in Germany Using Different Machine Learning Algorithms with AutoScout24 Dataset

This research paper focuses on predicting car prices in the German automotive market using machine learning algorithms and real-world vehicle data collected from the AutoScout24 platform.

The study highlights the importance of using real market data when building intelligent pricing systems. It explains how different vehicle specifications and technical characteristics influence vehicle prices and how machine learning algorithms can learn pricing patterns from historical data.

The paper compares multiple regression algorithms and analyzes their ability to predict vehicle prices accurately. It also emphasizes the importance of proper data cleaning, preprocessing, and feature selection before training machine learning models.

This research helped us better understand:

- Real-world automotive market behavior
- Vehicle pricing patterns
- Important pricing-related features
- Model comparison strategies
- Data preprocessing techniques
- Practical machine learning applications in business environments

The paper also reinforced the importance of building a system that is not only accurate from a technical perspective but also useful in real-world automotive pricing scenarios.

Research Paper Link

(PDF) Predicting Car Pricing in Germany using different Machine Learning Algorithms with AutoScout24 Dataset

1.7 Significance of the Project

The importance of this project extends to both academic and practical domains. From an academic perspective, it demonstrates the application of modern machine learning techniques in solving regression problems. From a business perspective, the proposed system can support customers, car dealerships, and online marketplaces by providing accurate and reliable price estimations.

Furthermore, the project highlights the potential of artificial intelligence in transforming traditional decision-making processes into automated, data-driven solutions.

1.8 Chapter Summary

This chapter introduced the problem of vehicle price estimation and discussed its impact on the automotive market. It also reviewed the research studies that addressed the same problem and presented the motivation, objectives, and significance of the proposed system. The following chapter focuses on understanding the dataset and analyzing the business aspects related to the project.

Chapter 2 : Dataset Understanding & Business Analysis

2.1 Dataset Overview

The dataset used in this project is a real-world automotive dataset designed for developing an intelligent car price prediction system using Machine Learning and Artificial Intelligence techniques. The dataset contains detailed information about used vehicles collected from automotive marketplace listings and includes multiple technical and market-related features that directly influence vehicle pricing.

The dataset was selected to support regression-based predictive modeling and to simulate realistic automotive market behavior. Each record in the dataset represents a single vehicle with its corresponding market price and technical specifications.

The dataset contains both numerical and categorical variables, allowing machine learning models to learn complex relationships between vehicle characteristics and pricing behavior. The diversity of the dataset improves the ability of the prediction models to generalize across different vehicle brands, models, fuel types, and market conditions.

The dataset includes important vehicle-related features such as:

- Vehicle price
- Model
- Color
- Registration date
- Engine power (KW and PS)

- Fuel type
- Transmission type
- Fuel consumption
- Mileage in kilometers

This diversity helps improve the robustness and reliability of the machine learning models by exposing them to different market segments ranging from economy vehicles to luxury and high-performance cars.

The dataset plays a critical role in the project because it serves as the foundation for:

- Exploratory Data Analysis (EDA)
- Data preprocessing
- Feature engineering
- Machine learning model training
- Hyperparameter tuning
- Performance evaluation
- Predictive analysis

Overall, the dataset represents a realistic automotive pricing environment and provides a strong foundation for building an intelligent AI-powered vehicle price prediction system.

2.2Dataset Source & Data Collection

The dataset used in this project was obtained from the publicly available automotive dataset:

“Used Car Price Prediction Dataset”

published on the Kaggle platform and designed for machine learning and predictive analytics applications related to vehicle price estimation.

Dataset source:

Used Car Price Prediction Dataset - Kaggle

The dataset was published by:

- **Author:** Taeef Najib

- **Platform:** Kaggle
- **Publication Date:** September 15, 2023

One of the main reasons for selecting this dataset is its strong academic and research relevance. **The dataset has been referenced and utilized in multiple published research papers** focusing on car price prediction using Machine Learning and Artificial Intelligence techniques.

According to the dataset source page, the dataset has already been cited in several academic studies indexed through Google Scholar, which reflects the dataset's credibility, research value, and suitability for predictive modeling tasks.

During the research phase of this project, multiple studies based on this dataset were reviewed and analyzed. From these studies, two research papers were selected as the primary academic foundation for this project because of their direct relevance to:

- Vehicle price prediction
- Regression analysis
- Ensemble learning techniques
- Boosting algorithms
- Feature engineering strategies
- Machine learning model comparison

The selected research papers helped guide the methodological and technical direction of the project, particularly in:

- Model selection
- Data preprocessing
- Feature engineering
- Hyperparameter tuning
- Performance evaluation

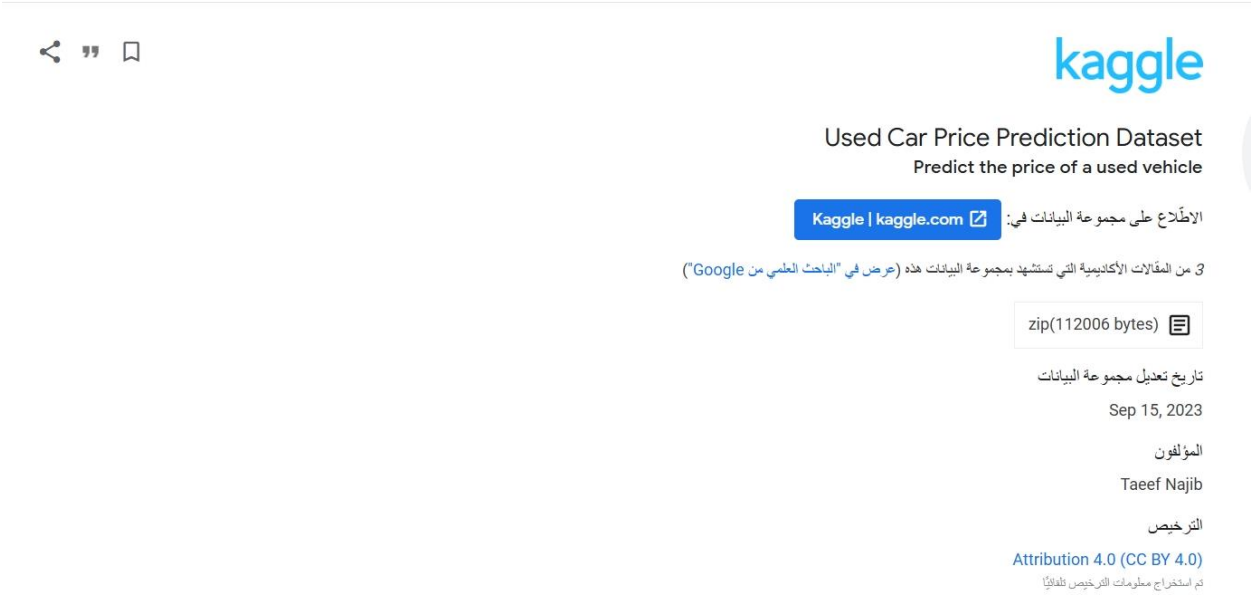
The first paper focused on comparing several regression and ensemble learning algorithms for vehicle price prediction, while the second paper emphasized predictive modeling using the AutoScout24 automotive dataset and advanced boosting techniques.

These research papers provided valuable insights into how modern machine learning algorithms can be applied effectively in intelligent automotive pricing systems and helped establish a strong scientific foundation for the implementation of this project.

The dataset itself contains structured vehicle information collected from real automotive marketplace listings. Each row represents a single vehicle, while each column represents a specific feature describing vehicle specifications, performance characteristics, and market-related information.

The collected data was later cleaned, transformed, and prepared for Machine Learning analysis through preprocessing and feature engineering techniques.

The following figure presents the original dataset source page from Kaggle used during the development of this project.



2.3 Dataset Importance

The dataset is considered one of the most important components of this project because the quality, diversity, and structure of the data directly affect the performance and accuracy of machine learning models.

Car price prediction is a complex regression problem influenced by many interconnected factors such as:

- Vehicle brand
- Vehicle model
- Vehicle age
- Mileage
- Fuel type
- Engine power
- Fuel consumption
- Transmission type

Because of this complexity, using a realistic and diverse dataset is essential for building a reliable intelligent pricing system.

The importance of the dataset in this project can be summarized as follows:

Supporting Accurate Price Prediction

The dataset provides real-world vehicle pricing information that helps machine learning algorithms learn actual market pricing behavior and generate accurate predictions.

Improving Machine Learning Performance

The availability of multiple vehicle specifications and pricing-related features allows the models to identify hidden relationships and improve predictive accuracy.

Enabling Feature Engineering

The dataset supports the creation of engineered features such as:

- Vehicle age
- Kilometers per year
- Log-transformed mileage

which help improve model understanding and prediction capability.

Representing Real Automotive Market Conditions

The dataset contains vehicles from different manufacturers, categories, and price ranges, making the project more realistic and applicable to real-world automotive pricing systems.

Supporting Business Decision-Making

The dataset helps simulate intelligent pricing systems used by:

- Car dealerships
- Automotive marketplaces
- Vehicle valuation systems
- Online car-selling platforms

where accurate pricing is essential for both buyers and sellers.

Enhancing AI-Based Analysis

The dataset enables the application of advanced Machine Learning and ensemble learning algorithms capable of analyzing complex nonlinear relationships within automotive market data.

Overall, the dataset provides the essential foundation required to design, train, evaluate, and deploy an intelligent AI-powered car price prediction system capable of generating reliable and data-driven vehicle price estimations.

2.4 Dataset Summary Statistics

General Information About the Dataset

Attribute	Description
Dataset Type	Structured Tabular Dataset
Domain	Automotive Market Analysis
Problem Type	Supervised Machine Learning
Task Type	Regression
Main Objective	Car Price Prediction
Target Variable	price
Number of Features	11 Features

2.5 Feature Description

#	Feature	Feature Type	Data Type	Description
1	price	Target Feature	Numerical	Represents the market selling price of the vehicle and is the main variable to be predicted using machine learning models.

2	brand	Vehicle Identification Feature	Categorical	Indicates the manufacturer or brand of the vehicle, such as BMW, Audi, Ford, or Hyundai.
3	model	Vehicle Identification Feature	Categorical	Represents the specific model of the vehicle, which helps differentiate between vehicle categories and market segments.
4	color	Appearance Feature	Categorical	Describes the exterior color of the vehicle, which may influence customer preference and resale attractiveness.
5	registration date	Temporal Feature	Categorical (Date)	Indicates the first registration date of the vehicle and is used to calculate vehicle age and depreciation.
6	power_kw	Technical Specification Feature	Numerical	Represents engine power measured in kilowatts (kW), reflecting the vehicle's technical performance.
7	power_ps	Technical Specification Feature	Numerical	Represents engine horsepower (PS), which measures engine strength and driving performance.
8	transmission type	Configuration Feature	Categorical	Describes the transmission system of the vehicle, such as Manual, Automatic, or Semi-Automatic.
9	fuel_type	Configuration Feature	Categorical	Indicates the type of fuel used by the vehicle, such as Petrol, Diesel, or Hybrid.
10	fuel_consumption_l_100km	Operational Feature	Numerical	Represents the amount of fuel consumed per 100 kilometers and measures fuel efficiency.
11	mileage_in_km	Operational Feature	Numerical	Indicates the total distance driven by the vehicle in kilometers and is an important factor affecting depreciation and resale value.

2.6 Feature Categories

Category	Features	Description
Target Feature	price	Represents the market selling price of the vehicle and serves as the dependent variable in the machine learning models.

Vehicle Identification Features	brand, model	These features identify the manufacturer and model of the vehicle and are essential for analyzing market segmentation and brand influence on pricing.
Technical Specification Features	power_kw, power_ps	Describe the engine performance and power output of the vehicle, which strongly affect vehicle value and driving performance.
Operational Features	fuel_consumption_l_100km, mileage_in_km	Represent vehicle usage and fuel efficiency, which directly influence depreciation, operating cost, and resale value.
Configuration Features	transmission_type, fuel_type	Describe the vehicle's mechanical and fuel configuration, affecting customer preferences, fuel economy, and pricing behavior.
Temporal Features	registration_date	Represents the vehicle registration date and is used to calculate vehicle age and depreciation over time.
Appearance Features	color	Describes the exterior color of the vehicle, which may influence customer attraction and resale appeal.

2.7 Data Preprocessing Explanation

Data preprocessing is considered one of the most critical stages in the Machine Learning lifecycle because the quality of the input data directly affects model accuracy, reliability, and overall predictive performance. Since real-world automotive datasets often contain inconsistencies, duplicate records, outliers, and non-optimized feature structures, several preprocessing and feature engineering techniques were applied to prepare the dataset for predictive modeling.

The preprocessing workflow implemented in this project focused on improving data quality, reducing noise, handling abnormal values, and creating meaningful features capable of enhancing machine learning model performance.

2.7.1 Importing Required Libraries

The project began by importing the required Python libraries used for:

- Data manipulation
- Numerical computation
- Data visualization
- Machine Learning modeling
- Model evaluation
- Date processing

The following libraries were used throughout the preprocessing and modeling stages:

Import libraries

In [ ]:
```
import numpy as np
import pandas as pd

import matplotlib.pyplot as plt
import seaborn as sns

from datetime import datetime

from sklearn.model_selection import train_test_split, RandomizedSearchCV
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.svm import SVR

from xgboost import XGBRegressor
from lightgbm import LGBMRegressor
from catboost import CatBoostRegressor

import warnings
warnings.filterwarnings('ignore')
```

These libraries provided the foundation for performing data preprocessing, exploratory analysis, feature engineering, model training, and performance evaluation.

2.7.2 Loading the Dataset

The dataset was loaded into a Pandas DataFrame using the `read_csv()` function. After loading the dataset, the dimensions of the dataset were examined to identify the total number of rows and columns available for analysis.

Load data

In [2]:
```
df = pd.read_csv(r"C:\Users\Lenovo\Desktop\Car Price Prediction\Car Price Prediction\Cars_Data.csv")
df.shape
```

Out[2]: (77119, 11)

2.7.3 Dataset Information Inspection

The dataset structure and feature data types were analyzed using the info() function to better understand:

- Feature names
- Data types
- Non-null values
- Dataset completeness

```
In [4]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 77119 entries, 0 to 77118
Data columns (total 11 columns):
 #   Column                     Non-Null Count  Dtype
---  ------                     --------------  -----
 0   price                      77119 non-null  int64
 1   brand                      77119 non-null  object
 2   model                      77119 non-null  object
 3   color                      77119 non-null  object
 4   registration_date          77119 non-null  object
 5   power_kw                   77119 non-null  int64
 6   power_ps                   77119 non-null  int64
 7   transmission_type          77119 non-null  object
 8   fuel_type                  77119 non-null  object
 9   fuel_consumption_l_100km.1  77119 non-null  float64
 10  mileage_in_km              77119 non-null  int64
dtypes: float64(1), int64(4), object(6)
memory usage: 6.5+ MB
```

This step played an important role in identifying:

- Numerical features
- Categorical features
- Date-related columns
- Potential missing values
- Data type conversion requirements

The extracted information guided the preprocessing strategy applied later in the project.

2.8 Data Cleaning

2.8.1 Duplicate Records Handling

Duplicate records can negatively affect machine learning model performance by introducing repeated patterns and bias into the dataset. Therefore, duplicate rows were identified and removed to improve dataset quality and ensure data consistency.

The total number of duplicate records was first checked using:

Remove duplicates

```
In [5]: df.duplicated().sum()

Out[5]: 2674
```

After identifying duplicate records, duplicates were removed using the following approach:

```
In [6]: df = df.drop_duplicates()
        df.shape

Out[6]: (74445, 11)
```

Removing duplicated entries helped improve the reliability of the dataset and reduced unnecessary redundancy during model training.

2.8.2 Cardinality Check

Cardinality analysis was performed to identify the number of unique values within each feature. This process helps understand feature diversity and supports decisions related to feature encoding and preprocessing.

The following code was used to calculate feature cardinality:

Cardinality check

```
In [7]: for col in df.columns:
            print(f"{col}: {df[col].nunique()}")

price: 10206
brand: 29
model: 510
color: 14
registration_date: 331
power_kw: 400
power_ps: 400
transmission_type: 4
fuel_type: 4
fuel_consumption_l_100km.1: 312
mileage_in_km: 28919
```

2.8.4 Logical Data Cleaning

Logical validation rules were applied to remove unrealistic or invalid records that could negatively affect model performance.

The following conditions were enforced:

Logical data cleaning

```
In [10]: df = df[df["mileage_in_km"] > 0]
         df = df[df["vehicle_age"] >= 0]
         df.shape

Out[10]: (74364, 11)
```

This cleaning process removed:

- Vehicles with invalid mileage values
- Vehicles with negative age values
- Inconsistent or logically incorrect records

Logical data cleaning helped improve data quality and ensured that the dataset represented realistic automotive market conditions.

2.8.5 Exploratory Data Analysis (EDA) — Target Distribution

Exploratory Data Analysis was performed to better understand the distribution of the target variable (`price`) before model training.

The following visualization was used:

Target distribution (EDA)

```
In [11]: plt.figure(figsize=(8,4))
         sns.histplot(df["price"], bins=50, kde=True)
         plt.title("Price Distribution")
         plt.show()
```

2.8.6 Outlier Handling Using Winsorization

Instead of removing outliers completely, the project applied Winsorization to reduce the influence of extreme price values while preserving valuable market information.

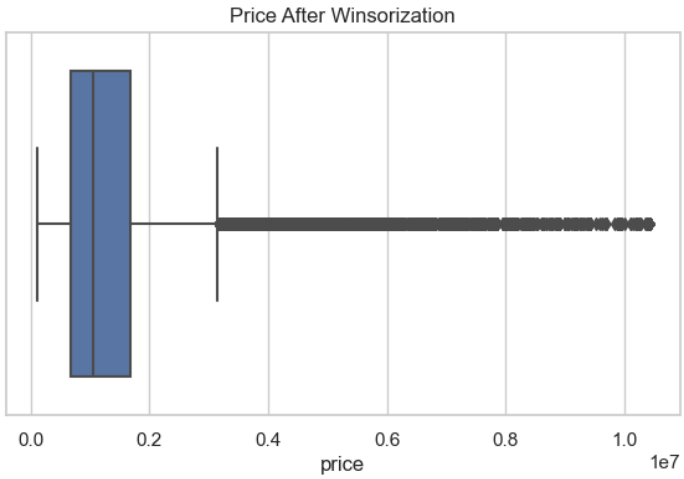
The following technique was used:

Outlier handling (WINSORIZATION, NOT deletion)

In [13]:
```
p_low, p_high = df["price"].quantile([0.01, 0.99])
df["price"] = df["price"].clip(p_low, p_high)
```

In [14]:
```
p_low, p_high = df["price"].quantile([0.01, 0.99])
df["price"] = df["price"].clip(p_low, p_high)
```

In [15]:
```
plt.figure(figsize=(7,4))
sns.boxplot(x=df["price"])
plt.title("Price After Winsorization")
plt.show()
```



Winsorization helped:

- Reduce the impact of extreme luxury vehicle prices
- Preserve important data points
- Improve model stability
- Reduce prediction variance
- Improve regression learning behavior

This approach was preferred over direct outlier deletion because luxury vehicles still represent valid market observations.

2.8.7 Log Transformation of Target Variable

The target variable (`price`) showed right-skewed distribution; therefore, logarithmic transformation was applied to normalize the distribution and improve regression model learning.

The following transformation was applied:

Log-transform target

In [16]:
```
df["log_price"] = np.log1p(df["price"])
```

In [17]:
```
plt.figure(figsize=(8,4))
sns.histplot(df["log_price"], bins=50, kde=True)
plt.title("Log-Price Distribution")
plt.show()
```

The log transformation helped:

- Reduce skewness
- Stabilize variance
- Improve regression performance
- Enhance model generalization
- Improve prediction consistency

This preprocessing technique is commonly used in real-world regression problems involving highly skewed financial data.

2.9 Feature Engineering Strategy

Feature Engineering is considered one of the most important stages in the Machine Learning pipeline because the quality of engineered features directly affects model learning capability and prediction performance. In this project, several feature engineering techniques were applied to transform raw automotive data into more meaningful and informative variables that better represent vehicle market behavior.

The main objective of the feature engineering process was to:

- Improve predictive performance
- Capture hidden relationships between variables
- Reduce data complexity
- Enhance model understanding
- Increase regression accuracy

The feature engineering strategy focused on transforming real-world automotive characteristics into machine learning-friendly representations capable of improving the intelligence of the pricing system.

2.9.1 Vehicle Age Engineering

The original registration_date feature was transformed into a numerical feature called vehicle_age.

This transformation was important because:

- Older vehicles generally have lower market value
- Vehicle depreciation strongly depends on age
- Numerical age representation is more suitable for regression models

The following transformation was applied:

Parse date → vehicle_age

In [8]:
```
df["registration_date"] = pd.to_datetime(
    df["registration_date"],
    format="%d-%m-%y",
    errors="coerce"
)

current_year = datetime.now().year
df["vehicle_age"] = current_year - df["registration_date"].dt.year

df.drop(columns=["registration_date"], inplace=True)
```

In [9]:
```
df["vehicle_age"].describe()
```

Out[9]:
```
count    74445.000000
mean         9.449204
std          4.926856
min          3.000000
25%          6.000000
50%          8.000000
75%         12.000000
max         31.000000
Name: vehicle_age, dtype: float64
```

2.9.3 Logarithmic Mileage Transformation

Mileage distribution within automotive datasets is usually highly skewed due to large variation between low-mileage and high-mileage vehicles. To reduce skewness and stabilize variance, logarithmic transformation was applied to the mileage feature.

Feature engineering

In [21]:
```
df["km_per_year"] = df["mileage_in_km"] / (df["vehicle_age"] + 1)
df["log_mileage"] = np.log1p(df["mileage_in_km"])
```

This transformation helped:

- Normalize mileage distribution

- Reduce the influence of extremely large mileage values
- Improve regression model stability

Logarithmic transformation is widely used in real-world Machine Learning regression problems involving financial and market-related variables.

2.9.5 Transmission Unknown Indicator Feature

The dataset contained vehicles with unknown transmission values. Instead of removing these records, a binary indicator feature was created to preserve potentially useful information.

Power features (remove redundancy)

```
In [20]: df["transmission_unknown_flag"] = (df["transmission_type"] == "Unknown").astype(int)
```

2.9.6 Feature Correlation Analysis

Feature relationships were analyzed to identify highly correlated variables and understand how numerical features interact with each other.

The following correlation analysis was performed:

Power features (remove redundancy)

```
In [18]: df[["power_kw", "power_ps"]].corr()
```

Out[18]:

	power_kw	power_ps
power_kw	1.000000	0.999996
power_ps	0.999996	1.000000

```
In [19]: df.drop(columns=["power_kw"], inplace=True)
```

2.10 Exploratory Data Analysis (EDA)

Exploratory Data Analysis (EDA) was performed to better understand the dataset structure, identify patterns, detect anomalies, and analyze relationships between vehicle features and pricing behavior.

EDA is considered a critical stage in Machine Learning projects because it provides valuable insights into:

- Data distribution
- Feature relationships
- Market behavior
- Outliers
- Data quality

The EDA process in this project focused on both numerical and categorical analysis to better understand the automotive market represented within the dataset.

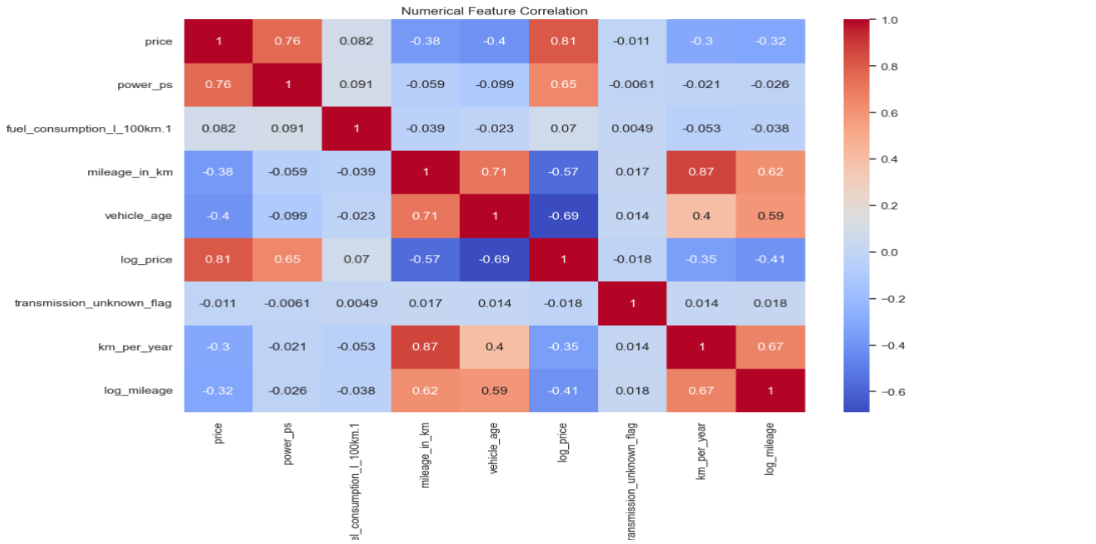
2.10.1 Numerical Correlation Heatmap

A numerical correlation heatmap was created to analyze the relationships between numerical variables and identify strong positive or negative correlations.

The following visualization was used:

```
In [22]: num_cols = df.select_dtypes(include=np.number).columns

plt.figure(figsize=(12,8))
sns.heatmap(df[num_cols].corr(),annot=True , cmap="coolwarm")
plt.title("Numerical Feature Correlation")
plt.show()
```



The correlation heatmap helped identify:

- Strong relationships between numerical features
- Potential multicollinearity
- Features highly related to vehicle pricing
- Redundant variables

The analysis revealed that:

- Engine power features showed strong positive correlation with vehicle price
- Mileage and vehicle age showed negative correlation with price
- Engine power measurements (power_kw and power_ps) were highly correlated

These findings helped guide feature engineering and model optimization decisions.

2.10.2 Categorical Feature Analysis

Categorical Exploratory Data Analysis was performed to analyze the distribution of categorical variables and identify the most frequent categories within the dataset.

The following visualization process was applied:

Categorical EDA

```
]: cat_cols = df.select_dtypes(include="object").columns

for col in cat_cols:
    plt.figure(figsize=(6,3))
    df[col].value_counts().head(10).plot(kind="bar")
    plt.title(f"Top 10 {col}")
    plt.show()
```

This analysis helped understand:

- Most common vehicle brands
- Most frequent vehicle models
- Fuel type distribution
- Transmission type distribution
- Market popularity trends

2.11 Business Understanding

The automotive industry depends heavily on accurate vehicle pricing because pricing directly affects:

- Sales performance
- Customer trust
- Market competitiveness
- Dealership profitability
- Buyer decision-making

Traditional vehicle pricing methods often rely on manual estimation, personal experience, or static market rules, which may lead to inconsistent and inaccurate price evaluations.

This project addresses this problem by developing an intelligent AI-powered pricing system capable of analyzing large amounts of automotive market data and generating reliable vehicle price predictions automatically.

The business understanding phase focused on identifying:

- Factors affecting vehicle prices
- Consumer market behavior
- Vehicle depreciation patterns
- Brand influence on pricing
- Usage impact on resale value

Key business insights identified during analysis include:

- Luxury brands maintain significantly higher market prices
- Vehicle age strongly affects resale value
- Lower mileage vehicles generally achieve higher prices
- Fuel-efficient and hybrid vehicles show growing market value
- Engine performance influences pricing especially in luxury and sports segments

The developed system can support:

- Car dealerships
- Online automotive marketplaces
- Vehicle valuation companies
- Automotive analytics platforms
- Buyers and sellers

by providing intelligent and data-driven price estimations.

2.12 SWOT Analysis

SWOT analysis was performed to evaluate the strengths, weaknesses, opportunities, and threats related to the proposed AI-powered car price prediction system.

Strengths	**Weaknesses**
Uses real-world automotive market data	Dataset may not cover all global markets
High prediction accuracy using advanced ML models	Some features may contain limited information
Supports intelligent automated pricing	Market conditions change continuously

Strengths	Weaknesses
Ensemble learning improves model performance	Luxury vehicle imbalance may affect learning
Feature engineering enhances predictive capability	Requires continuous model updating
Opportunities	**Threats**
Growing demand for AI-driven automotive solutions	Rapid market fluctuations
Expansion into online vehicle marketplaces	Competitor pricing systems
Integration with dealership platforms	Data inconsistency across regions
Future deployment as a web application	Economic conditions affecting vehicle markets
Real-time vehicle valuation services	Changes in fuel and automotive regulations

The SWOT analysis demonstrated that the project has strong practical and commercial potential within intelligent automotive pricing systems.

2.13 Market Analysis

The automotive market is one of the largest and most competitive industries worldwide, with vehicle pricing playing a critical role in both consumer decisions and business operations.

Modern automotive marketplaces increasingly depend on:

- Data analytics
- Artificial Intelligence
- Predictive systems
- Intelligent recommendation engines

to improve pricing accuracy and market efficiency.

Several important market trends were identified during analysis:

Increasing Demand for AI-Based Pricing

Automotive companies and online marketplaces are increasingly adopting Machine Learning technologies to automate pricing and improve valuation accuracy.

Growth of Online Automotive Platforms

Digital vehicle marketplaces require intelligent systems capable of generating instant and reliable price estimations.

Importance of Data-Driven Decision Making

Automotive businesses are shifting toward data-driven pricing strategies instead of relying solely on manual expertise.

Expansion of Hybrid and Fuel-Efficient Vehicles

The growing popularity of hybrid and fuel-efficient vehicles is changing automotive pricing behavior and market demand.

High Competition in Used Vehicle Markets

Used car markets require accurate pricing systems to maintain competitiveness and customer trust.

This project aligns with these market trends by providing a scalable AI-powered pricing solution capable of supporting modern intelligent automotive systems and future digital marketplace applications.

Chapter 3 : System Analysis

3.1 System Requirements

The proposed system is designed as an intelligent AI-powered web platform that predicts vehicle prices based on automotive specifications and market-related features. The system combines Machine Learning capabilities with a user-friendly web interface to provide fast, accurate, and reliable car price estimations.

The platform is intended to serve multiple types of users including:

- Individual car buyers
- Vehicle sellers
- Car dealerships
- Automotive analysts
- Online vehicle marketplaces

The main purpose of the system is to simplify the vehicle valuation process by replacing manual estimation methods with an automated intelligent prediction system.

The system requirements were identified based on:

- Automotive market needs
- User expectations
- AI prediction requirements
- Website usability standards
- Real-world pricing challenges

The platform must provide:

- Fast prediction performance
- High prediction accuracy
- Smooth user interaction
- Secure data handling
- Scalable architecture
- Reliable AI integration

The system is designed to operate as an interactive web application where users can enter vehicle information and instantly receive an estimated market price generated by the trained Machine Learning models.

3.2 Functional Requirements

Functional requirements describe the core operations and services that the system must perform to satisfy user and business needs.

The proposed AI car price prediction system should provide the following functionalities:

3.2.1 Vehicle Price Prediction

The system must allow users to enter vehicle specifications such as:

- Brand
- Model
- Fuel type
- Transmission type
- Mileage

- Vehicle age
- Engine power
- Vehicle color

After submitting the information, the system should generate an estimated vehicle price using the trained Machine Learning model.

3.2.2 Smart Search Engine

The platform should include a smart search engine that allows users to quickly search for:

- Vehicle brands
- Car models
- Similar vehicles
- Vehicle categories

The search functionality should improve navigation efficiency and user experience.

3.2.3 Real-Time Prediction Response

The system must generate predictions within a very short response time to ensure smooth user interaction and fast decision-making.3.2.4 User-Friendly Interface

The website should provide:

- Easy navigation
- Clear input forms
- Organized layout
- Responsive design
- Simple interaction process

The interface should support users with different technical backgrounds.

3.2.5 Data Validation

The system should validate user input before prediction generation to avoid invalid or unrealistic vehicle data.

Examples include:

- Negative mileage values

- Invalid vehicle age
- Missing required fields

3.2.6 Machine Learning Model Integration

The platform must integrate the trained AI model with the backend system to allow automatic prediction generation directly through the website.

3.2.7 Prediction Result Display

The predicted vehicle price should be displayed clearly with organized formatting to help users understand the output easily.

3.2.8 Future Scalability Support

The system architecture should support future improvements such as:

- Advanced recommendation systems
- Vehicle comparison tools
- Real-time market analysis
- User accounts and saved searches
- Integration with online marketplaces

3.3 Non-Functional Requirements

Non-functional requirements describe the quality standards and operational characteristics of the proposed system.

3.3.1 Performance

The system should provide:

- Fast prediction speed
- Smooth website performance
- Minimal loading time
- Efficient AI model execution

The prediction process should be completed within seconds.

3.3.2 Accuracy

The AI system must maintain high prediction accuracy using advanced Machine Learning models such as:

- CatBoost
- XGBoost
- LightGBM

Accurate predictions are critical for building user trust and supporting real-world pricing decisions.

3.3.3 Usability

The website should be:

- Simple to use
- Easy to understand
- Accessible for non-technical users
- Visually organized

The user experience should remain smooth across all pages and prediction steps.

3.3.4 Scalability

The system should support future expansion including:

- Larger datasets
- More users
- Additional AI models
- New automotive features
-

3.3.5 Reliability

The system should produce stable and consistent predictions without failures during normal operation.

3.3.6 Security

The platform should protect:

- User input data
- Backend communication
- Model interaction processes

Basic web security practices should be applied to ensure safe system usage.

3.3.7 Maintainability

The system should be designed in a modular and organized way to simplify:

- Future updates
- Bug fixing
- Model replacement
- Feature enhancement

3.4 User Requirements

The proposed platform is designed to satisfy the needs of users who require quick and reliable vehicle price estimation.

Users expect the system to:

- Be simple and easy to use
- Provide fast responses
- Deliver accurate price predictions
- Support different vehicle types
- Work smoothly on desktop and mobile devices

Users should be able to:

- Enter vehicle specifications easily
- Search for vehicle information quickly
- Receive understandable prediction results
- Navigate the platform without technical complexity

The system must reduce the time and effort required for manual vehicle valuation and help users make smarter automotive decisions.

3.5 System Workflow

The workflow of the proposed system describes how data moves through the platform from user input to final prediction output.

Step 1: User Access

The user opens the web platform through a browser.

Step 2 : Vehicle Information Input

The user enters vehicle specifications including:

- Brand
- Model
- Fuel type
- Mileage
- Transmission type
- Vehicle age
- Engine power

Step 3: Input Validation

The system validates the entered data to ensure correctness and completeness.

Step 4: Data Preprocessing

The backend system applies preprocessing and feature engineering operations including:

- Encoding
- Scaling
- Feature transformation
- Engineered feature generation

Step 5 : AI Prediction

The processed data is passed to the trained Machine Learning model which generates the predicted vehicle price.

Step 6 : Result Display

The estimated price is displayed to the user through the website interface.

Step 7: User Decision Support

The prediction result helps the user:

- Evaluate vehicle value

- Compare prices
- Support buying or selling decisions

3.6 User Roles

The system currently focuses on general public users while supporting future administrative expansion.

3.6.1 General User

General users can:

- Access the website
- Enter vehicle information
- Generate price predictions
- Use the search functionality
- View estimated prices

This role represents:

- Buyers
- Sellers
- Dealership customers
- Automotive enthusiasts

3.6.2 Administrator (Future Expansion)

Future versions of the platform may include administrative roles responsible for:

- Managing datasets
- Monitoring system performance
- Updating Machine Learning models

3.7 AI System Requirements

The AI component represents the core intelligence of the proposed platform. The Machine Learning system must satisfy several technical and operational requirements to ensure high-quality prediction performance.

3.7.1 High Prediction Accuracy

The AI model should provide highly accurate vehicle price estimations capable of handling real-world automotive market variations.

3.7.2 Support for Complex Relationships

The system should detect nonlinear relationships between vehicle features and market prices.

Advanced boosting models such as:

- CatBoost
- LightGBM
- XGBoost

were selected because of their strong capability in handling complex structured datasets.

3.7.3 Fast Inference Speed

The trained AI model must generate predictions quickly to support real-time website interaction.

3.7.4 Large Dataset Compatibility

The AI system should efficiently process large automotive datasets without major performance degradation.

3.7.5 Feature Engineering Support

The AI pipeline should support engineered features such as:

- Vehicle age
- Mileage per year
- Logarithmic transformations (to improve predictive performance.)

3.7.6 Continuous Improvement Capability

The system should support future retraining and optimization using updated automotive market data.

This ensures that the prediction model remains aligned with changing market trends and vehicle pricing behavior.

Chapter 4 : System Modeling & Diagrams

This chapter presents the system modeling and workflow diagrams used in the development of the AI-based Car Price Prediction System. These diagrams provide a visual representation of how the system operates, how users interact with the platform, and how machine learning processes are executed internally.

The modeling phase was essential for understanding system behavior, organizing workflow logic, and simplifying communication between system components during development.

4.1 Use Case Diagram

The Use Case Diagram illustrates the interaction between users, administrators, and the AI prediction system. It represents the major functionalities supported by the platform, including data preprocessing, model training, evaluation, deployment, and vehicle price prediction.

The diagram demonstrates how the system allows users to:

- Enter vehicle details
- Validate input data
- Request price predictions
- Use the trained AI model

It also illustrates the administrative and machine learning operations including:

- Uploading datasets
- Data preprocessing
- Model training
- Model evaluation
- Best model selection
- Model deployment

The use case structure helps clarify the responsibilities of each actor and the sequence of system interactions.

Main Actors in the System

1. User

The user interacts with the website to:

- Enter car specifications
- Submit prediction requests
- Receive estimated vehicle prices

2. Administrator / Data Scientist

The administrator manages the machine learning workflow including:

- Dataset management
- Model training
- Model evaluation
- Hyperparameter tuning
- Model deployment

3. AI Prediction System

The AI engine processes user input, applies the trained model, and generates accurate vehicle price predictions.

Core Use Cases

Use Case	Description
Enter Car Details	Users input vehicle specifications into the system
Validate Input	The system verifies input correctness and completeness
Get Price Prediction	The AI model predicts the estimated vehicle price
Upload Dataset	Dataset is uploaded for model training
Preprocess Data	Cleaning, transformation, and feature engineering operations are applied
Train Models	Machine learning models are trained using historical data
Evaluate Models	Models are compared using evaluation metrics
Select Best Model	The highest-performing model is selected
Deploy Model	Final model is prepared for real-world usage

Use Case Diagram Figure

4.2 Activity Diagram

The Activity Diagram illustrates the complete workflow of the machine learning pipeline used in the project, beginning from dataset loading and ending with final model evaluation.

The workflow was designed to ensure:

- Clean data preparation
- Efficient preprocessing
- Proper feature engineering
- Reliable model training
- Accurate performance evaluation

The activity flow also demonstrates the separation between preprocessing techniques used for traditional machine learning models and CatBoost.

Workflow Explanation

The system workflow starts by loading the dataset and performing initial preprocessing operations such as:

- Removing duplicates
- Cleaning invalid records
- Parsing date features
- Generating engineered features

After preprocessing, the workflow proceeds to:

- Train-test splitting
- Categorical preprocessing
- Model training
- Hyperparameter tuning
- Final evaluation

The diagram also illustrates the use of:

- One-Hot Encoding for traditional machine learning models
- Native categorical handling for CatBoost

Finally, the tuned models are evaluated using:

- MAE

- RMSE
- R^2 Score

The workflow concludes by comparing training and testing performance to ensure model generalization and minimize overfitting.

Main Workflow Stages

1. Data Preparation

This phase includes:

- Loading the dataset
- Removing duplicates
- Removing invalid observations
- Feature engineering
- Winsorization
- Log transformation

2. Preprocessing

This phase handles:

- Train/test split
- Categorical encoding
- Data preparation for different models

3. Modeling Phase

This phase includes:

- Training baseline models
- Generating predictions
- Comparing model performance

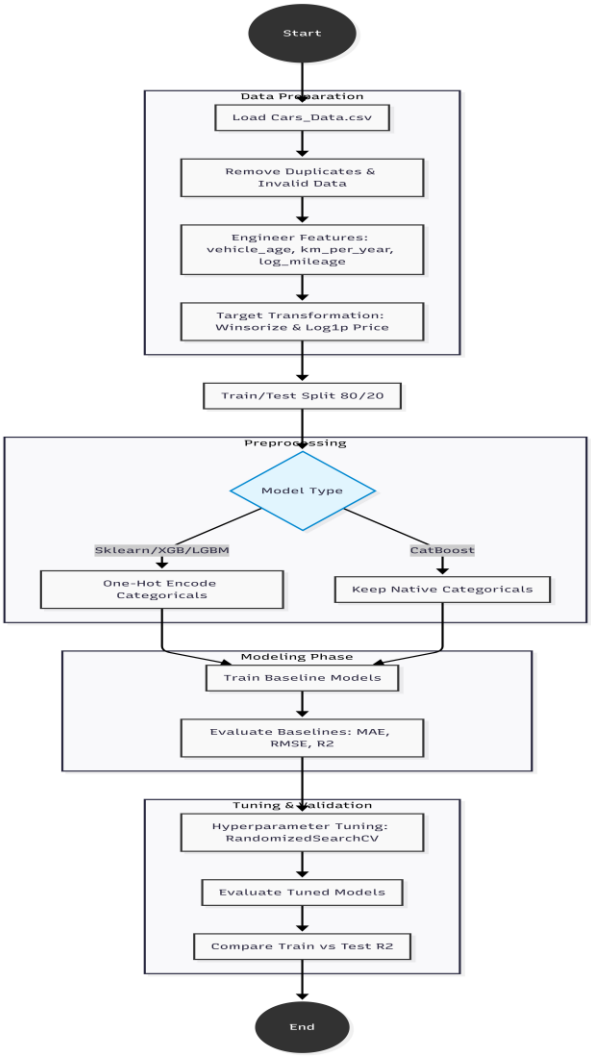
4. Tuning & Validation

This phase includes:

- RandomizedSearchCV hyperparameter tuning

- Tuned model evaluation
- Overfitting analysis
- Final model selection

Activity Diagram Figure



4.3 Sequence Diagrams

Sequence diagrams are used to describe the interaction between different components inside the Car Price Prediction System over time. These diagrams illustrate how users, administrators, APIs, machine learning models, and the database communicate to perform prediction and model management operations.

The system contains two main sequence diagrams:

1. **Admin Sequence Diagram**
2. **User Sequence Diagram**

These diagrams help explain the internal workflow of the system and provide a clear understanding of the communication process between the frontend, backend, machine learning engine, and storage components.

4.3.1 Admin Sequence Diagram

The Admin Sequence Diagram explains how the administrator interacts with the system to upload datasets, retrain machine learning models, evaluate performance, and deploy the best-performing model into production.

The process starts when the admin uploads a new dataset through the admin dashboard. The dataset is then sent to the backend API and stored in the database. After that, the administrator triggers the model retraining process.

Inside the machine learning pipeline, the system performs preprocessing operations such as:

- Removing duplicate records
- Cleaning invalid data
- Encoding categorical features
- Splitting the dataset into training and testing sets

After preprocessing, the system starts hyperparameter tuning using `RandomizedSearchCV` to improve model performance. The trained models are evaluated using MAE, RMSE, and R^2 metrics.

If the model achieves the required performance threshold, the system stores the trained model artifacts and allows deployment. Otherwise, the system sends a low-performance alert to the administrator.

Admin Sequence Diagram

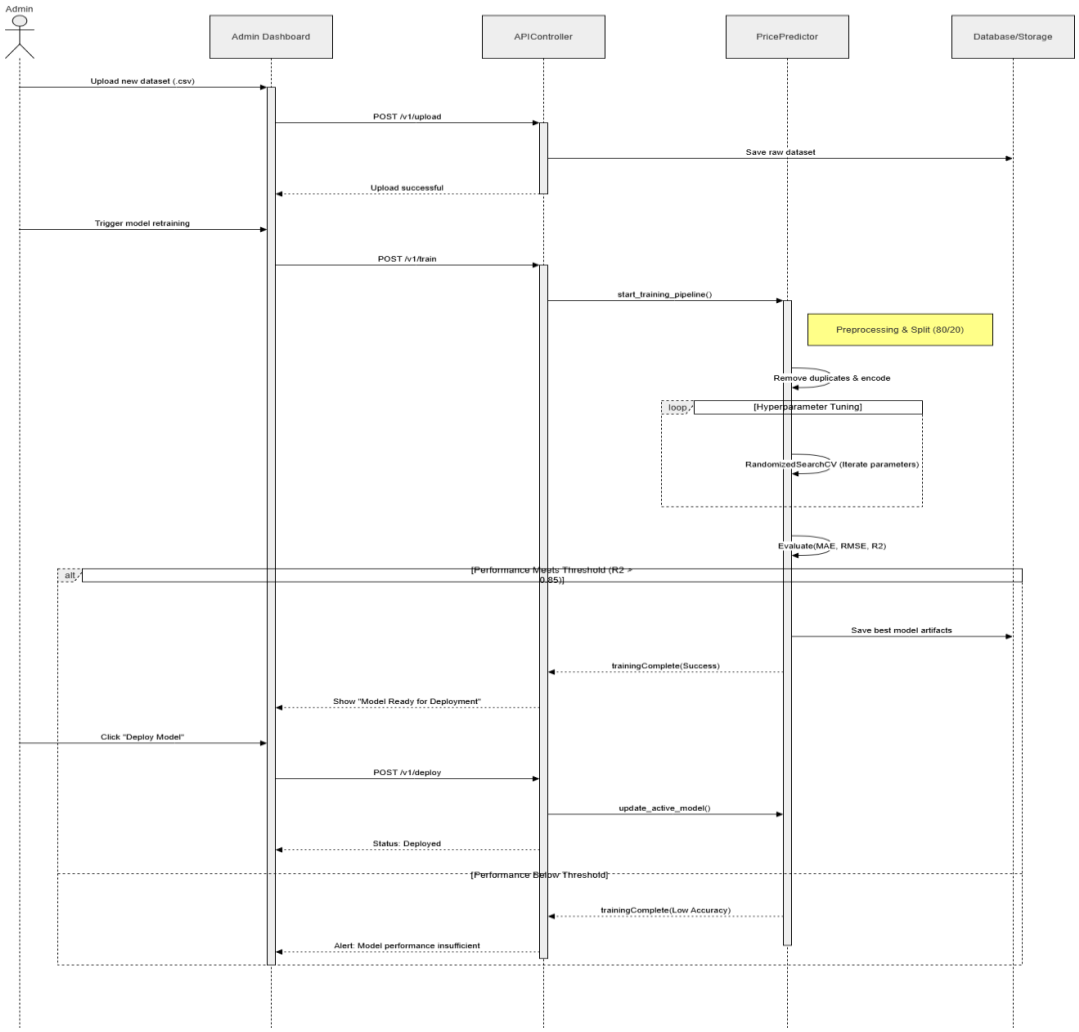


Figure Description

The figure demonstrates the complete interaction flow between the Admin Dashboard, API Controller, Machine Learning Prediction Engine, and Database during dataset upload, model training, evaluation, and deployment operations.

4.3.2 User Sequence Diagram

The User Sequence Diagram explains how end users interact with the AI-powered car price prediction system to obtain predicted car prices.

The process begins when the user enters car details through the web application interface developed using React. The frontend first validates the entered information to ensure that all required fields are correct and complete.

If invalid data is detected, validation error messages are displayed directly to the user. Otherwise, the frontend sends the request to the FastAPI backend using an HTTP POST request.

The backend API forwards the request to the prediction engine, where the trained CatBoost model and preprocessing artifacts are loaded. The system then performs feature engineering operations, including:

- Vehicle age calculation
- Mileage-per-year calculation
- Logarithmic feature transformation

After preprocessing the input data, the machine learning model generates the predicted car price and sends the result back to the frontend interface.

Finally, the predicted price is displayed to the user in a simple and user-friendly format. In case of server-side issues, the system returns an error message informing the user that the service is temporarily unavailable.

This workflow ensures fast, reliable, and accurate prediction results while maintaining a smooth user experience.

Importance of Sequence Diagrams

The sequence diagrams provide a detailed visualization of how data and requests move through the system. They help in:

- Understanding backend and frontend communication
- Explaining AI model integration
- Clarifying system logic and workflow
- Simplifying debugging and future maintenance
- Demonstrating real-world system interaction behavior

User Sequence Diagram

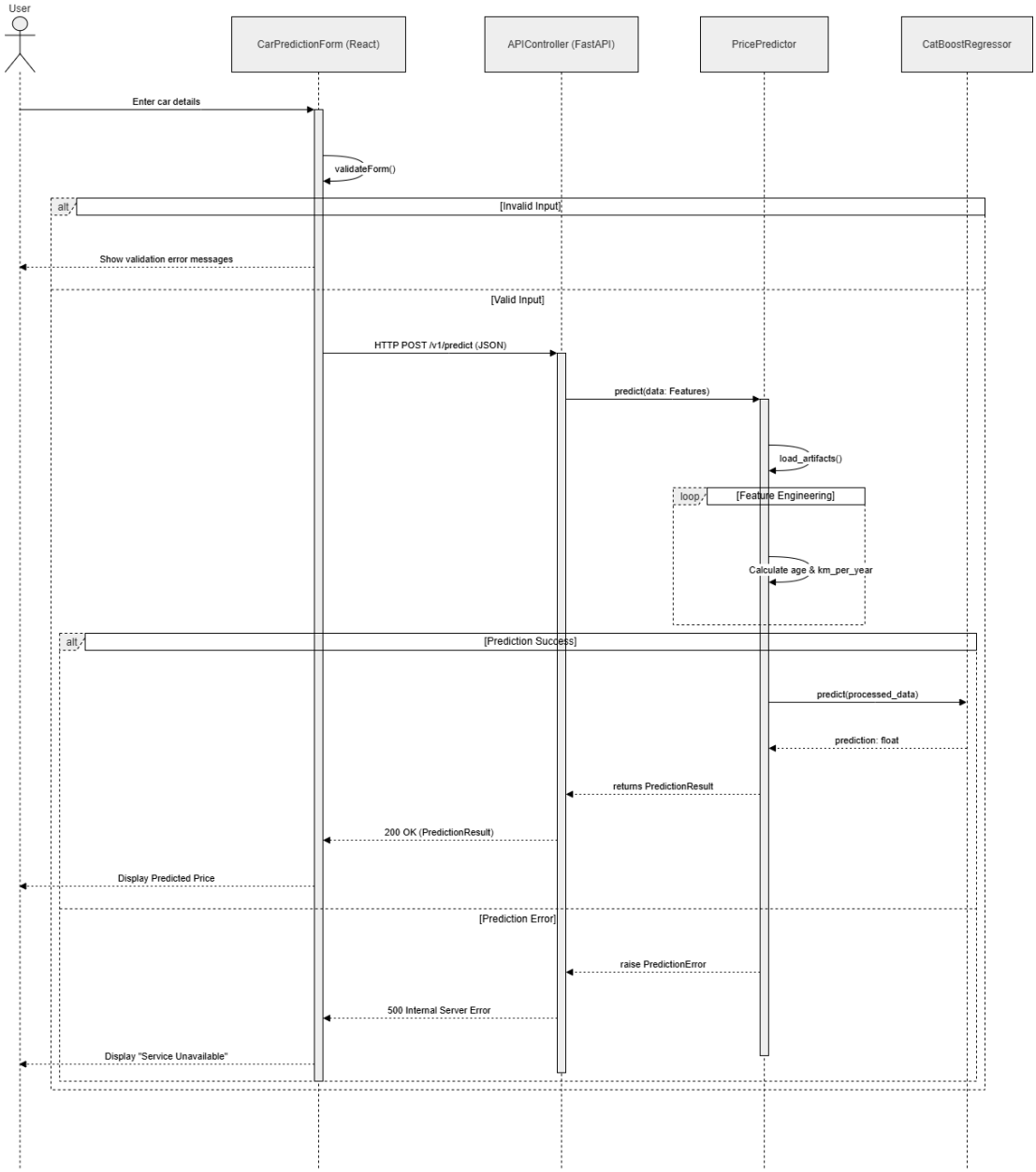


Figure Description

The figure illustrates the interaction between the user interface, FastAPI backend, prediction engine, and CatBoost model during the car price prediction process.

4.4 Class Diagram

The Class Diagram presents the static structure of the Car Price Prediction System and illustrates the relationships between the frontend interface, backend services, data transfer objects, and machine learning prediction components.

The purpose of this diagram is to provide a detailed view of the software architecture by identifying the main classes, their attributes, methods, and interactions within the system.

The system follows a layered architecture consisting of three primary layers:

- Frontend Layer (React + Tailwind CSS)
- Backend Layer (FastAPI)
- Machine Learning Service Layer (CatBoost Prediction Engine)

The frontend layer is responsible for collecting user inputs and displaying prediction results. The backend layer handles request validation, API communication, and prediction processing. The machine learning service layer manages model loading, feature transformation, and price prediction generation.

The class structure was designed to ensure modularity, maintainability, scalability, and clear separation of responsibilities among system components.

Class Diagram

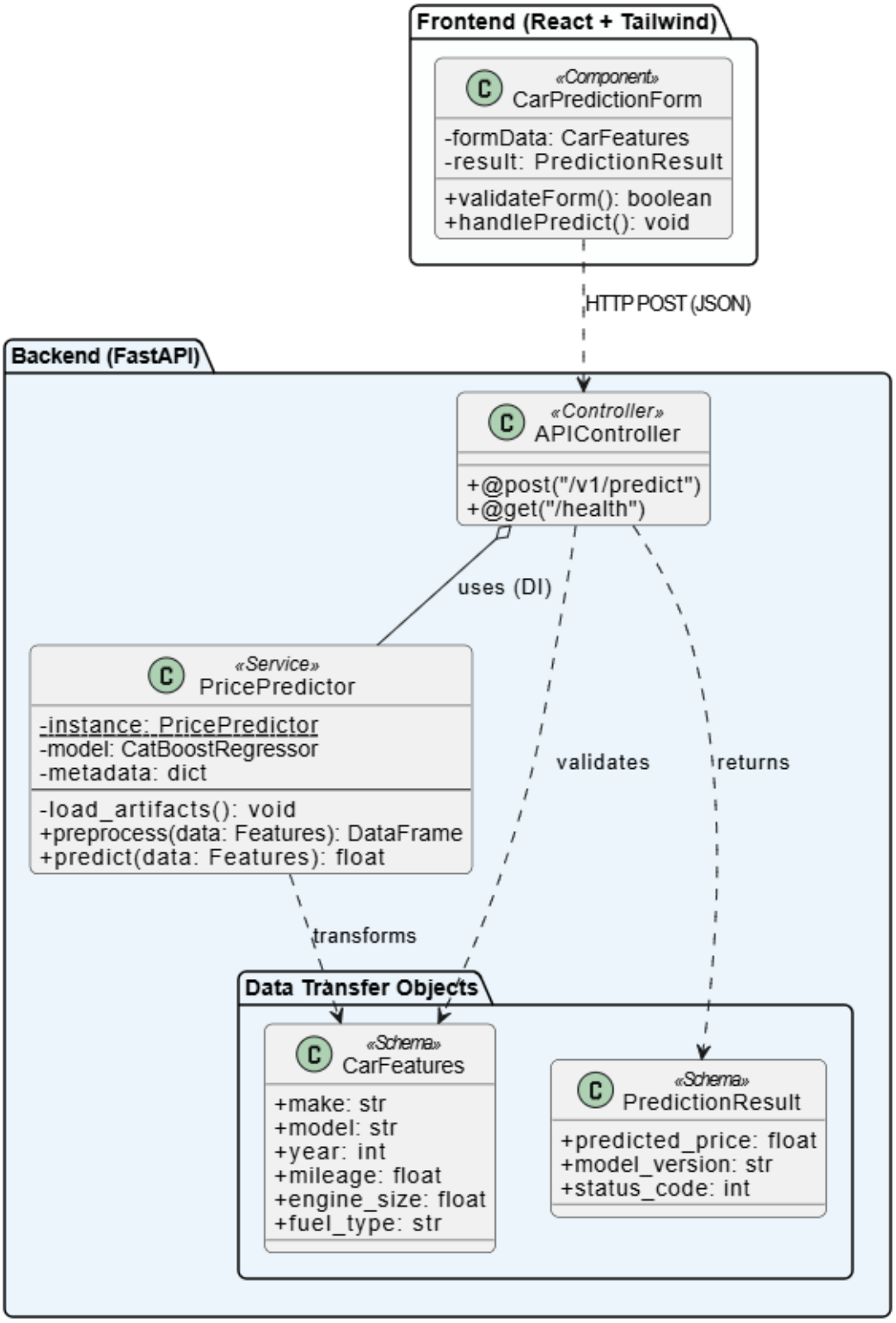


Figure Description

The Class Diagram contains the following major components:

1. CarPredictionForm (Frontend Component)

This component represents the user interface where vehicle information is entered.

Responsibilities:

- Collect user input
- Validate form data
- Submit prediction requests
- Display prediction results

Main Attributes

- formData : CarFeatures
- result : PredictionResult

Main Methods

- validateForm()
- handlePredict()

2. APIController (Backend Controller)

The API controller acts as the communication bridge between the frontend application and the machine learning prediction service.

Responsibilities:

- Receive prediction requests
- Validate incoming data
- Call prediction services
- Return prediction responses

Main Endpoints

- POST /v1/predict
- GET /health

3. PricePredictor (Prediction Service)

This is the core machine learning service responsible for processing input data and generating predictions.

Responsibilities:

- Load trained model artifacts
- Apply feature engineering
- Transform input data
- Generate predicted prices

Main Attributes

- model : CatBoostRegressor
- metadata : dictionary

Main Methods

- load_artifacts()
- preprocess()
- predict()

4. CarFeatures (Data Transfer Object)

This class represents the vehicle information provided by the user.

Example Attributes

- make
- model
- year
- mileage
- engine_size
- fuel_type

These attributes are transferred from the frontend to the backend and used as input for prediction.

5. PredictionResult (Response Object)

This class stores the prediction output generated by the machine learning model.

Attributes

- predicted_price
- model_version
- status_code

The object is returned to the frontend and displayed to the user.

Importance of the Class Diagram

The Class Diagram helps stakeholders understand:

- The internal structure of the application
- Object relationships and dependencies
- Responsibilities of each system component
- Data flow between layers
- Future extensibility and maintenance opportunities

This diagram serves as the blueprint for the software implementation phase and ensures consistency between design and development.

4.5 Deployment Diagram

The Deployment Diagram illustrates how the Car Price Prediction System is physically deployed and how its components interact within the production environment.

The architecture follows a modern web-based deployment model that separates the presentation layer, application layer, machine learning layer, and storage layer.

This design improves system scalability, maintainability, security, and performance while enabling efficient communication between users and the AI prediction engine.

Deployment Diagram

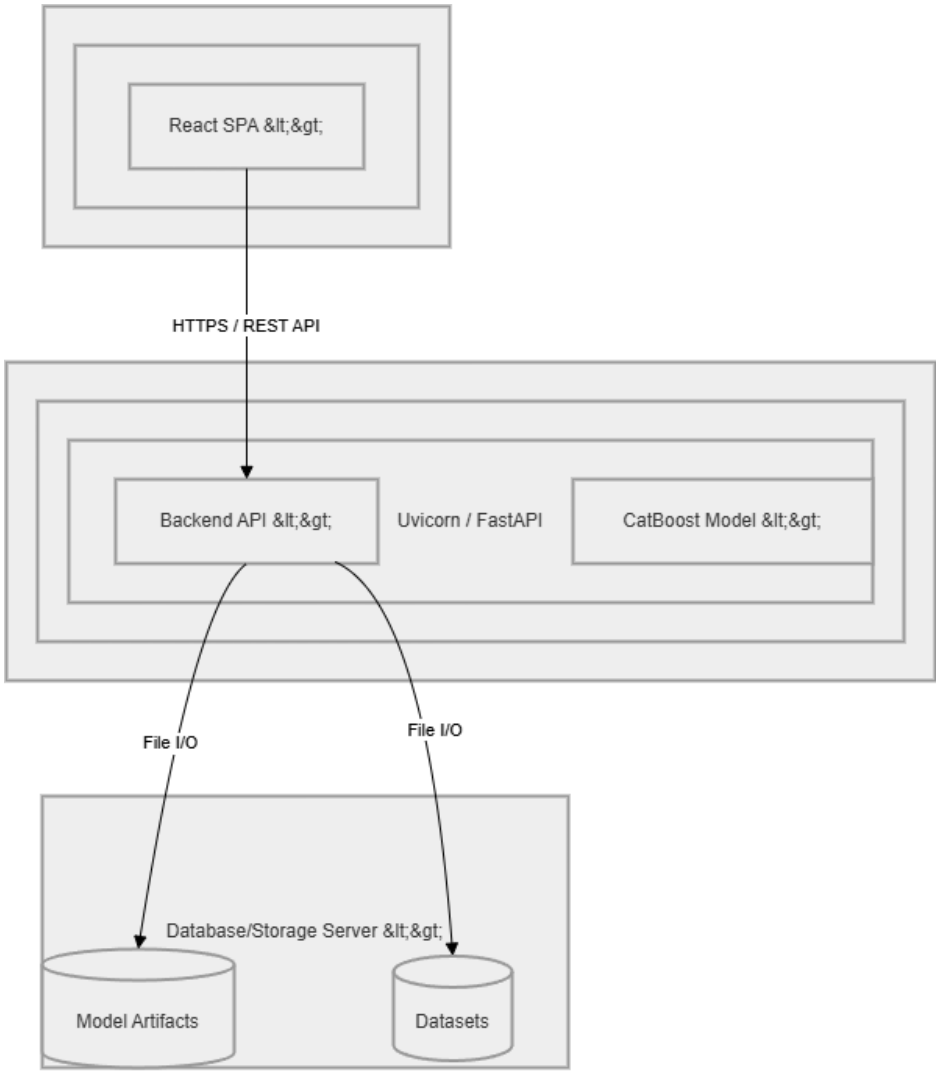


Figure Description

The deployment architecture consists of four major components:

1. Frontend Layer

Technology Used

- React.js
- Tailwind CSS

The frontend layer provides an interactive web interface where users enter vehicle information and receive predicted prices.

Responsibilities

- User interaction
- Form validation
- API communication
- Result visualization

The frontend communicates securely with the backend through HTTPS requests.

2. Backend Layer

Technology Used

- FastAPI
- Uvicorn

The backend serves as the central processing unit of the application.

Responsibilities

- Receive API requests
- Validate user inputs
- Manage prediction workflow
- Return prediction responses

The backend acts as the bridge between the user interface and the machine learning model.

3. Machine Learning Layer

Technology Used

- CatBoost Regressor

The machine learning layer contains the trained prediction model selected after extensive experimentation and hyperparameter tuning.

Responsibilities

- Load trained model artifacts
- Perform feature engineering
- Process incoming vehicle data
- Generate accurate price predictions

The CatBoost model was selected because it achieved the highest predictive performance among all tested models.

4. Storage Layer

The storage layer contains all persistent system resources.

Stored Components

- Trained model artifacts
- Dataset files
- Metadata files
- Feature engineering configurations

The backend accesses these resources through file-based storage operations during prediction and model management processes.

Deployment Workflow

The deployment process follows the sequence below:

1. The user enters vehicle information through the React web application.
2. The frontend sends an HTTPS request to the FastAPI backend.
3. The backend validates the received data.
4. The prediction service loads the trained CatBoost model.
5. Feature engineering transformations are applied.
6. The model generates the predicted car price.
7. The prediction result is returned to the backend.
8. The backend sends the response to the frontend.
9. The predicted price is displayed to the user.

Importance of the Deployment Diagram

The Deployment Diagram provides a clear understanding of:

- System infrastructure architecture
- Communication between components
- Model deployment strategy
- Data storage mechanisms
- Production environment design
- Scalability and maintainability considerations

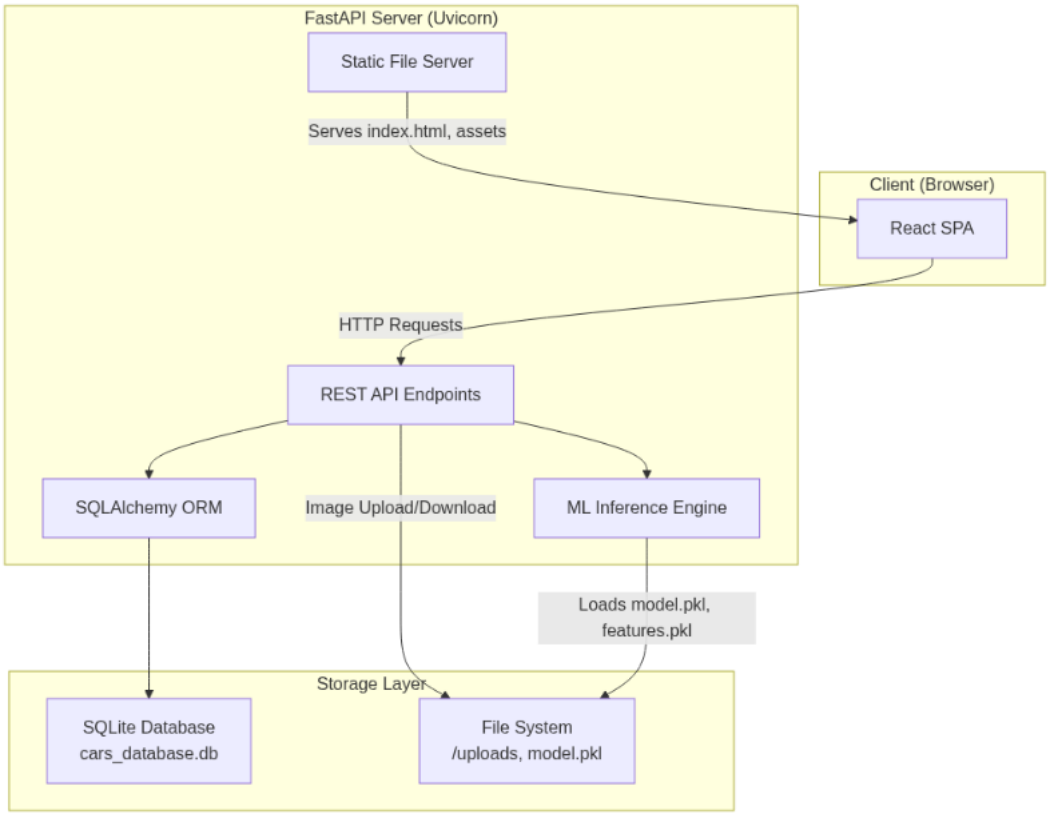
The diagram demonstrates how the complete AI-powered Car Price Prediction System operates in a real-world deployment environment and how its components collaborate to deliver fast and accurate predictions to end users.

Chapter 5 — System Design

5.1 System Architecture

The Car Price Prediction system follows a **monolithic, two-tier client-server architecture** where the backend serves both the API and the compiled frontend as a single deployment unit.

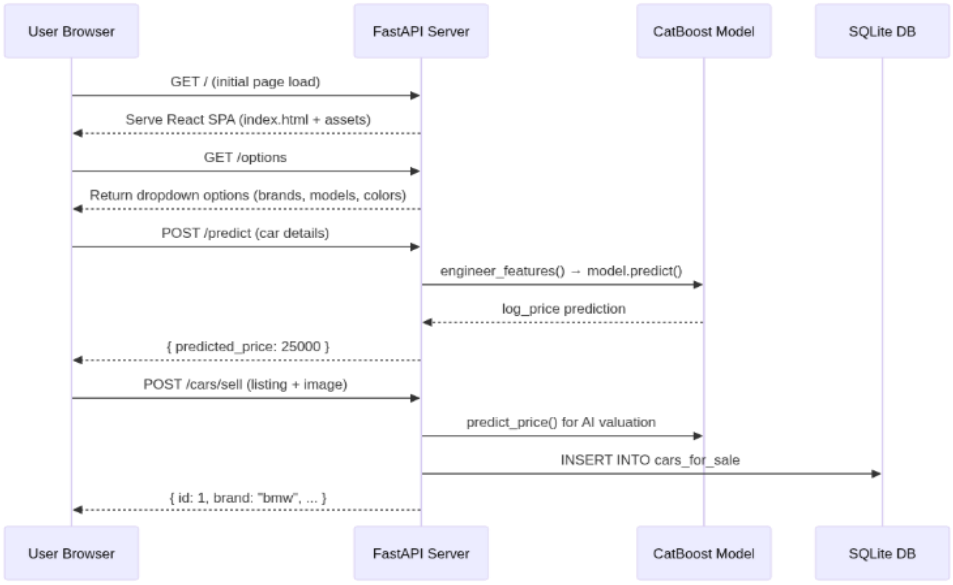
High-Level Architecture



Architecture Decisions

Decision	Choice	Rationale
Architecture Pattern	Monolithic (Backend serves Frontend)	Simplified deployment — single server serves everything
Communication	REST over HTTP (JSON)	Simple, stateless, widely supported
Frontend Delivery	Pre-built SPA served by FastAPI	Eliminates need for separate frontend server in production
ML Integration	In-process model loading at startup	Low-latency predictions, no inter-process overhead
Database	Embedded SQLite	Zero-config, file-based, ideal for the project scale

Component Interaction Flow

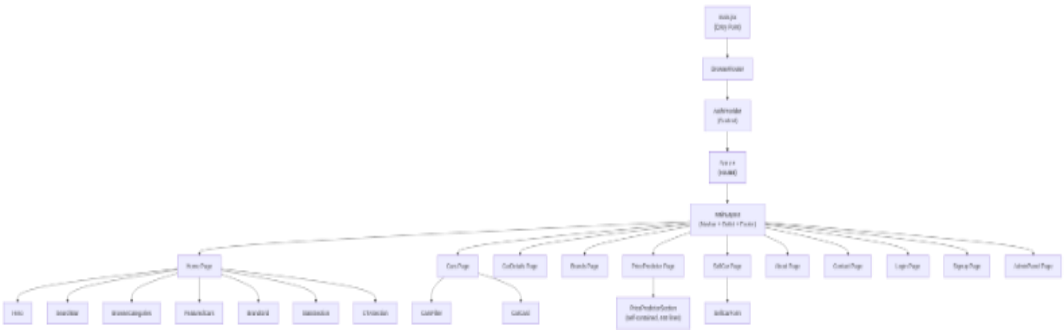


5.2 Frontend Design

Technology Stack

Technology	Version	Purpose
React	^18.3.1	UI component library
Vite	^5.4.8	Build tool and dev server
Tailwind CSS	^3.4.0	Utility-first CSS framework
React Router DOM	^6.26.1	Client-side routing (SPA)
Lucide React	^0.445.0	Icon library
PostCSS + Autoprefixer	^8.5.14 / ^10.5.0	CSS processing

Component Architecture



Frontend Directory Structure

```
frontend/
├── index.html                  # HTML entry point
```

```
├── package.json                # Dependencies & scripts
├── vite.config.js              # Vite + React plugin config
├── tailwind.config.js          # Tailwind theme customization
├── postcss.config.js           # PostCSS plugins
├── public/                     # Static public assets
├── dist/                       # Production build output
└── src/
    ├── main.jsx                # React DOM render + providers
    ├── App.jsx                 # Route definitions
    ├── App.css                 # Global styles
    ├── index.css               # Tailwind base imports
    ├── assets/                 # Images (logo.png, etc.)
    ├── components/             # Reusable UI components (12 files)
    │   └── sellCar/            # Sell car form (1 file)
    ├── context/
    │   └── AuthContext.jsx     # Authentication state management
    ├── data/                   # Static data (cars, userListings)
    ├── hooks/                  # Custom React hooks
    ├── layouts/
    │   └── MainLayout.jsx      # Navbar + content + Footer wrapper
    ├── pages/                  # 11 page components
    └── services/
        └── predictorApi.js     # API client for /predict & /cars/sell
```

State Management

State Type	Solution	Usage
Authentication	React Context (`AuthContext`)	User session, login/signup/logout
Token Storage	`localStorage`	JWT token persistence across sessions
Component State	`useState` hooks	Form inputs, UI toggles, loading states
Side Effects	`useEffect` hooks	API calls on mount, token validation

Design System

The website follows a **monochromatic grayscale** aesthetic with no brand accent color. All primary actions, buttons, and accents use dark gray/black tones.

Token	Value	Usage
Primary (Dark)	`#111827` / `gray-900`	CTA buttons, badges, navbar, footer background
Primary Hover	`#000000` / `black`	Button hover states
Section Background	`#f5f5f7`	Hero, Brands, About, Admin, CarDetails sections
Card Background	`#FFFFFF` / `white`	Cards, content areas, main layout
Text Primary	`gray-900`	Headings, body text
Text Secondary	`gray-500` / `gray-600`	Subtext, nav links, descriptions
Text Muted	`gray-400`	Footer links, placeholders
Surface	`gray-50` / `gray-100`	Input backgrounds, tag chips, spec boxes
Border	`gray-100` / `gray-200`	Card borders, dividers
CTA Gradient	`from-[#0f172a] via-[#111827] to-[#1e293b]`	CTA section dark gradient
Glow Accent	`blue-500/10`, `blue-500/20`	Subtle decorative background blurs

Shadow (Soft)	`shadow-sm`, `shadow-xl`, `shadow-2xl`	Card elevation, button depth
Border Radius	`rounded-2xl` (1rem), `rounded-[32px]`, `rounded-[40px]`	Buttons, cards, sections

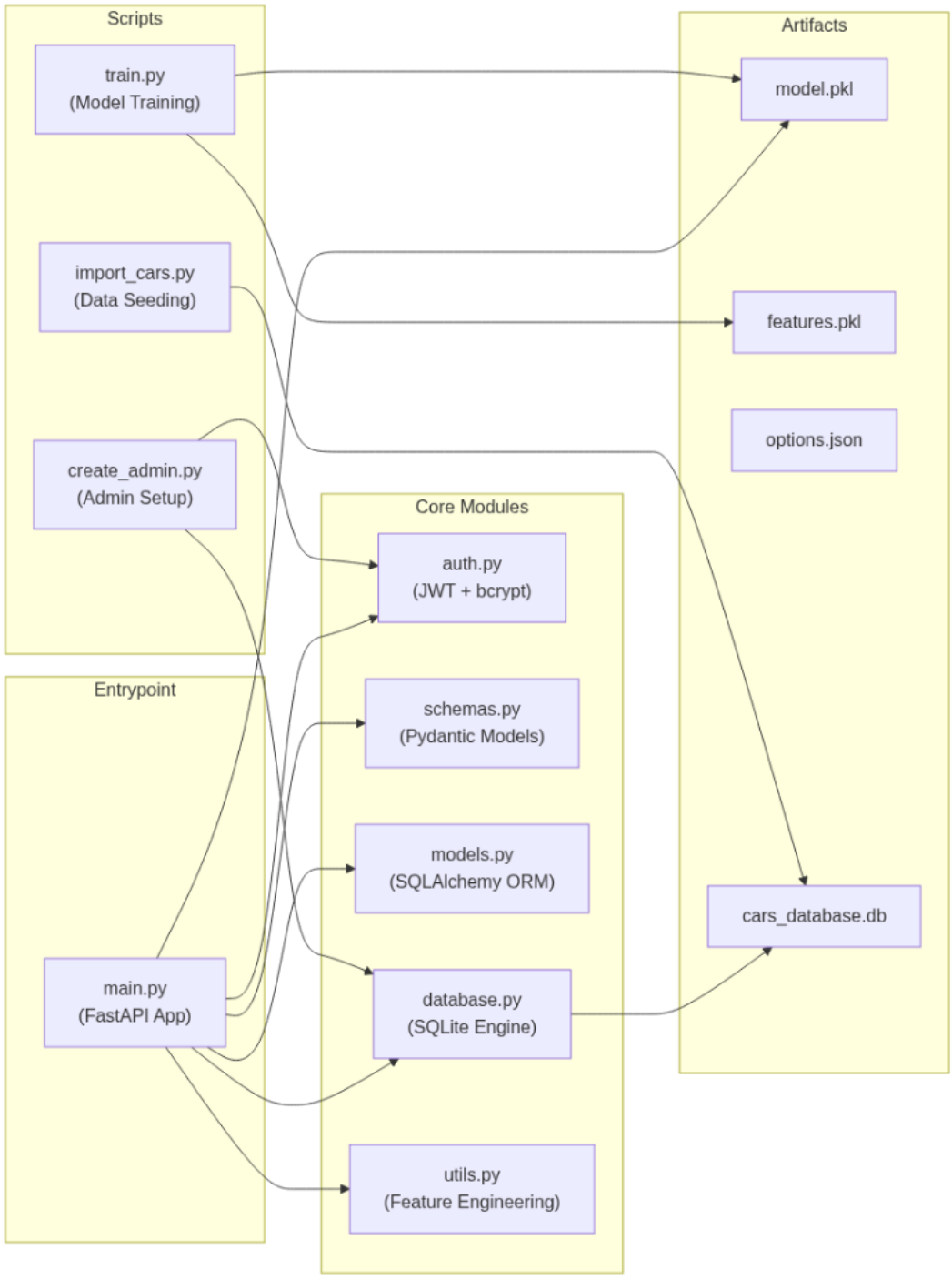
---

5.3 Backend Design

Technology Stack

Technology	Purpose
FastAPI	Web framework (async-capable, auto-docs)
Uvicorn	ASGI server
SQLAlchemy	ORM for database operations
Pydantic	Request/response validation schemas
Joblib	Model serialization/deserialization
bcrypt	Password hashing
PyJWT	JSON Web Token encoding/decoding
python-multipart	Multipart form data (file uploads)

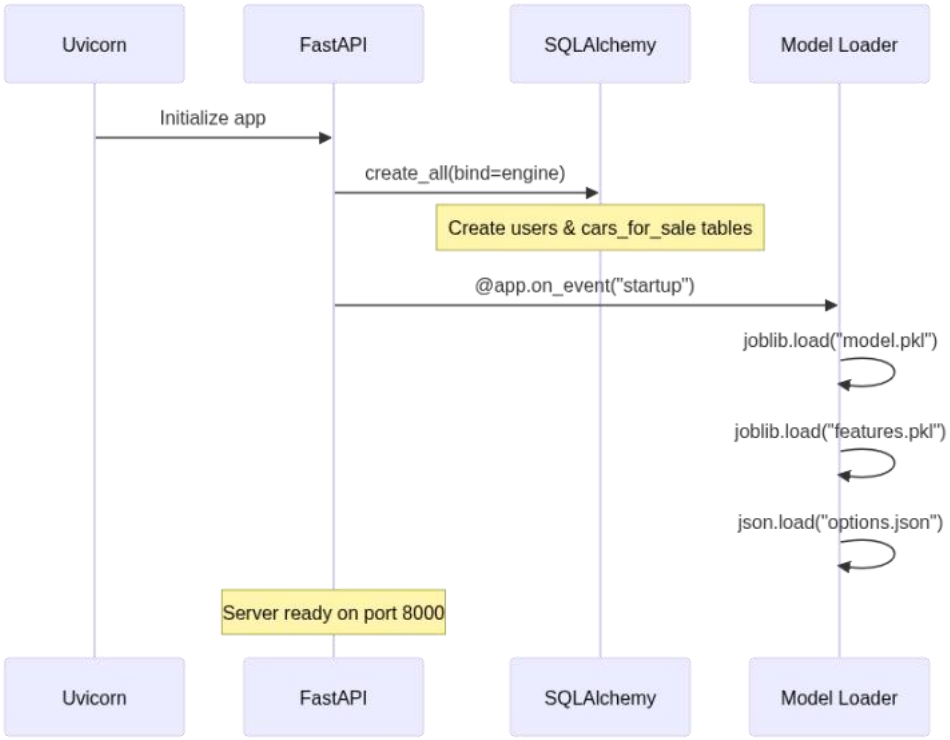
1.1.11 Backend Module Architecture



Backend Directory Structure

```
backend/
├── main.py              # FastAPI app, all endpoints, startup, static serving
├── auth.py              # Password hashing (bcrypt) and JWT token management
├── database.py          # SQLAlchemy engine, session factory, get_db dependency
├── models.py            # ORM models: User, CarForSale
├── schemas.py           # Pydantic schemas for API validation
├── utils.py             # Feature engineering pipeline + predict_price()
├── train.py             # CatBoost training script (data → model.pkl)
├── create_admin.py      # One-time admin account creation script
├── import_cars.py       # Database seeding with sample car listings
├── model.pkl            # Serialized CatBoost model (~4.2 MB)
├── features.pkl         # Feature list used during training
├── options.json         # Dropdown options (brands, models, colors, etc.)
├── cars_database.db     # SQLite database file
├── requirements.txt     # Python dependencies
└── uploads/             # User-uploaded car images
```

Startup Sequence



5.4 Machine Learning Architecture

Model Overview

Attribute	Detail
Algorithm	CatBoostRegressor

Task	Regression (used car price prediction)
Target Variable	`log_price` = log1p(price)
Inverse Transform	`expm1(log_price)` → actual price
Training Data	`Cars_Data.csv` (~5.2 MB)
Random Seed	42
Test Split	80/20 (train/test)

Training Pipeline

Feature Engineering

Feature	Type	Source	Engineering
power_ps	Numerical	Direct input	Raw horse power

`fuel_consumption_l_100km.1`	Numerical	`fuel_consumption`	Mapped from user input
`mileage_in_km`	Numerical	`mileage`	Mapped from user input
`vehicle_age`	Numerical	`registration_year`	`current_year - registration_year`
`transmission_unknown_flag`	Numerical	`transmission_type`	`1` if "Unknown", else `0`
`km_per_year`	Numerical	Derived	`mileage / (vehicle_age + 1)`
`log_mileage`	Numerical	Derived	`log1p(mileage)`
`brand`	Categorical	Direct input	e.g., "ford", "bmw"
`model`	Categorical	Direct input	e.g., "Kuga", "X5"
`color`	Categorical	Direct input	e.g., "black", "silver"
`transmission_type`	Categorical	Direct input	Automatic, Manual, Semi-automatic, Unknown
`fuel_type`	Categorical	Direct input	Petrol, Diesel, Electric, Hybrid

Total features: 12 (7 numerical + 5 categorical)

Inference Pipeline

1.1.12 Data Preprocessing Steps

1. **Duplicate removal** — Drop identical rows
2. **Date parsing** — `registration_date` (DD-MM-YY) → `vehicle_age`
3. **Column dropping** — Remove `power_kw` (perfectly correlated with `power_ps`)
4. **Logical filtering** — Remove rows where mileage ≤ 0 or vehicle_age < 0
5. **Outlier handling** — Winsorize price at 1st and 99th percentiles
6. **Log transformation** — `log1p(price)` as regression target to handle right-skewed distribution

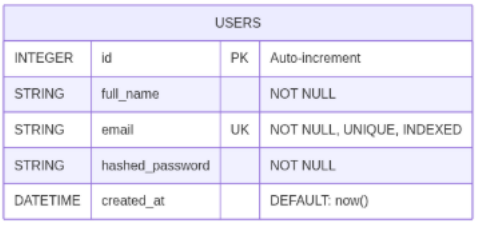
5.5 Database Design

Database Engine

Property	Value
DBMS	SQLite 3
File	`backend/cars_database.db`
ORM	SQLAlchemy 2.x
Connection	`sqlite:///./cars_database.db`

Config	`check_same_thread: False` (required for FastAPI)
Session	Per-request via `get_db()` dependency injection

Entity-Relationship Diagram



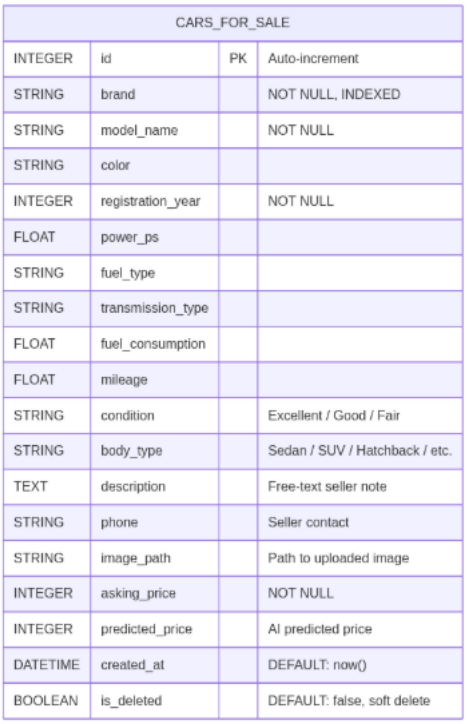


Table: `users`

Column	Type	Constraints	Description
`id`	INTEGER	PK, Auto-increment, Indexed	Unique user ID
`full_name`	STRING	NOT NULL	User's display name
`email`	STRING	NOT NULL, UNIQUE, Indexed	Login email
`hashed_password`	STRING	NOT NULL	bcrypt hashed password
`created_at`	DATETIME	DEFAULT `now()`	Registration timestamp

Table: `cars_for_sale`

Column	Type	Constraints	Description
`id`	INTEGER	PK, Auto-increment, Indexed	Unique listing ID
`brand`	STRING	NOT NULL, Indexed	Car brand (e.g., "bmw")
`model_name`	STRING	NOT NULL	Car model (e.g., "X5")
`color`	STRING	—	Car color
`registration_year`	INTEGER	NOT NULL	Year of first registration
`power_ps`	FLOAT	—	Engine power in PS

`fuel_type`	STRING	—	Petrol, Diesel, Electric, Hybrid
`transmission_type`	STRING	—	Automatic, Manual, Semi-automatic
`fuel_consumption`	FLOAT	—	L/100km
`mileage`	FLOAT	—	Odometer reading in km
`condition`	STRING	—	Excellent, Good, Fair
`body_type`	STRING	—	Sedan, SUV, Hatchback, Coupe, etc.
`description`	TEXT	—	Free-text from seller
`phone`	STRING	—	Seller phone number
`image_path`	STRING	—	Relative URL to uploaded image
`asking_price`	INTEGER	NOT NULL	User's asking price
`predicted_price`	INTEGER	NULLABLE	AI-predicted market price
`created_at`	DATETIME	DEFAULT `now()`	Listing creation timestamp
`is_deleted`	BOOLEAN	DEFAULT `false`, NOT NULL	Soft-delete flag for admin trash

---

5.6 API Design

Base Configuration

Property	Value
Framework	FastAPI
Base URL	http://127.0.0.1:8000
Data Format	JSON (except file uploads: multipart/form-data)
Auth Method	Bearer Token (JWT in `Authorization` header)
CORS	All origins allowed (`*`)
Auto Documentation	Swagger UI at `/docs`, ReDoc at `/redoc`

Endpoint Summary

Method	Endpoint	Auth	Description
`GET`	`/health`	—	Server health + model status
`GET`	`/options`	—	Dropdown options for prediction form
`POST`	`/predict`	—	AI price prediction
`POST`	`/signup`	—	User registration
`POST`	`/login`	—	User login → JWT token
`GET`	`/users/me`	Bearer	Get current user profile
`POST`	`/cars/sell`	—	Create car listing (with AI prediction + image)
`GET`	`/cars`	—	List all active car listings

DELETE	/cars/{car_id}	Admin	Soft-delete a car listing
GET	/admin/cars/deleted	Admin	List soft-deleted cars (trash)
POST	/admin/cars/{car_id}/restore	Admin	Restore a soft-deleted car
GET	/uploads/{filename}	—	Serve uploaded car images
GET	/{catchall:path}	—	Serve React SPA (frontend)

Detailed Endpoint Specifications

`POST /predict`

Purpose: Run AI price prediction on car attributes.

Request Body (JSON):

```
{
  "brand": "ford",
  "model": "Kuga",
  "color": "black",
  "registration_year": 2018,
  "power_ps": 140,
  "fuel_type": "Petrol",
  "transmission_type": "Automatic",
  "fuel_consumption": 6.5,
  "mileage": 50000
}
```

Response (200):

```
{
  "predicted_price": 25000
}
```

Errors: `400` (invalid input), `503` (model not loaded)

`POST /signup`

Request Body:

```
{
  "full_name": "John Doe",
  "email": "john@example.com",
  "password": "secret123"
}
```

Response (200): `{ "id": 1, "full_name": "John Doe", "email": "john@example.com" }`

Errors: `400` (email already registered)

`POST /login`

Request Body:

```
{
  "email": "john@example.com",
  "password": "secret123"
}
```

Response (200): `{ "access_token": "eyJ...", "token_type": "bearer" }`

Errors: `400` (incorrect email or password)

`POST /cars/sell`

Content-Type: `multipart/form-data`

Field	Type	Required
brand	string	☑
model_name	string	☑
color	string	☑
registration_year	int	☑
power_ps	float	☑
fuel_type	string	☑
transmission_type	string	☑
fuel_consumption	float	☑
mileage	float	☑
asking_price	int	☑
condition	string	✗
body_type	string	✗
description	string	✗
phone	string	✗
image	file	✗

Response (200): Full `CarResponse` object including `predicted_price` from AI.

`DELETE /cars/{car_id}`

Auth: Admin only (email must be admin@tara.com)

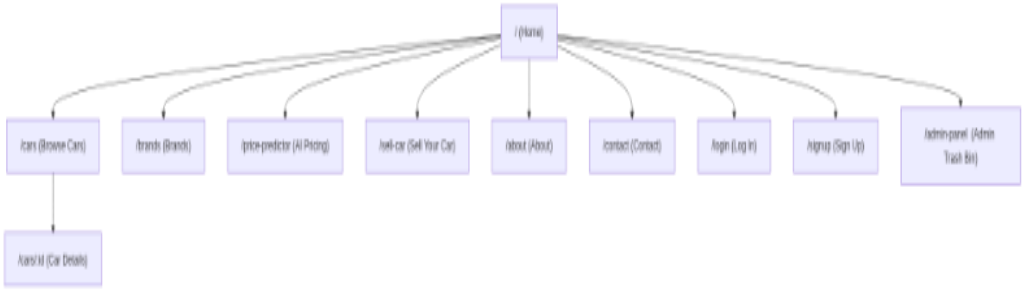
Response (200): `{ "message": "Car moved to trash successfully" }`

Pydantic Schema Definitions

Schema	Usage	Fields

`PredictionRequest`	POST /predict input	brand, model, color, registration_year, power_ps, fuel_type, transmission_type, fuel_consumption, mileage
`PredictionResponse`	POST /predict output	predicted_price
`UserCreate`	POST /signup input	full_name, email, password
`UserLogin`	POST /login input	email, password
`UserResponse`	User profile output	id, full_name, email
`Token`	POST /login output	access_token, token_type
`CarResponse`	Car listing output	All 17 car fields + id

5.7 Website Map



Page Descriptions

Route	Page	Description	Auth Required
`/`	Home	Landing page: Hero, Search, Categories, Featured Cars, Brands, Stats, CTA	No
`/cars`	Cars	Browse all car listings with filters (brand, body type, price range, fuel, transmission)	No
`/cars/:id`	Car Details	Individual car listing with full specs, images, price comparison, contact	No
`/brands`	Brands	Grid of available car brands	No
`/price-predictor`	AI Pricing	Interactive form to get AI price prediction for any car	No
`/sell-car`	Sell Your Car	Multi-field form to list a car for sale (with image upload)	No
`/about`	About	About the platform, team info	No

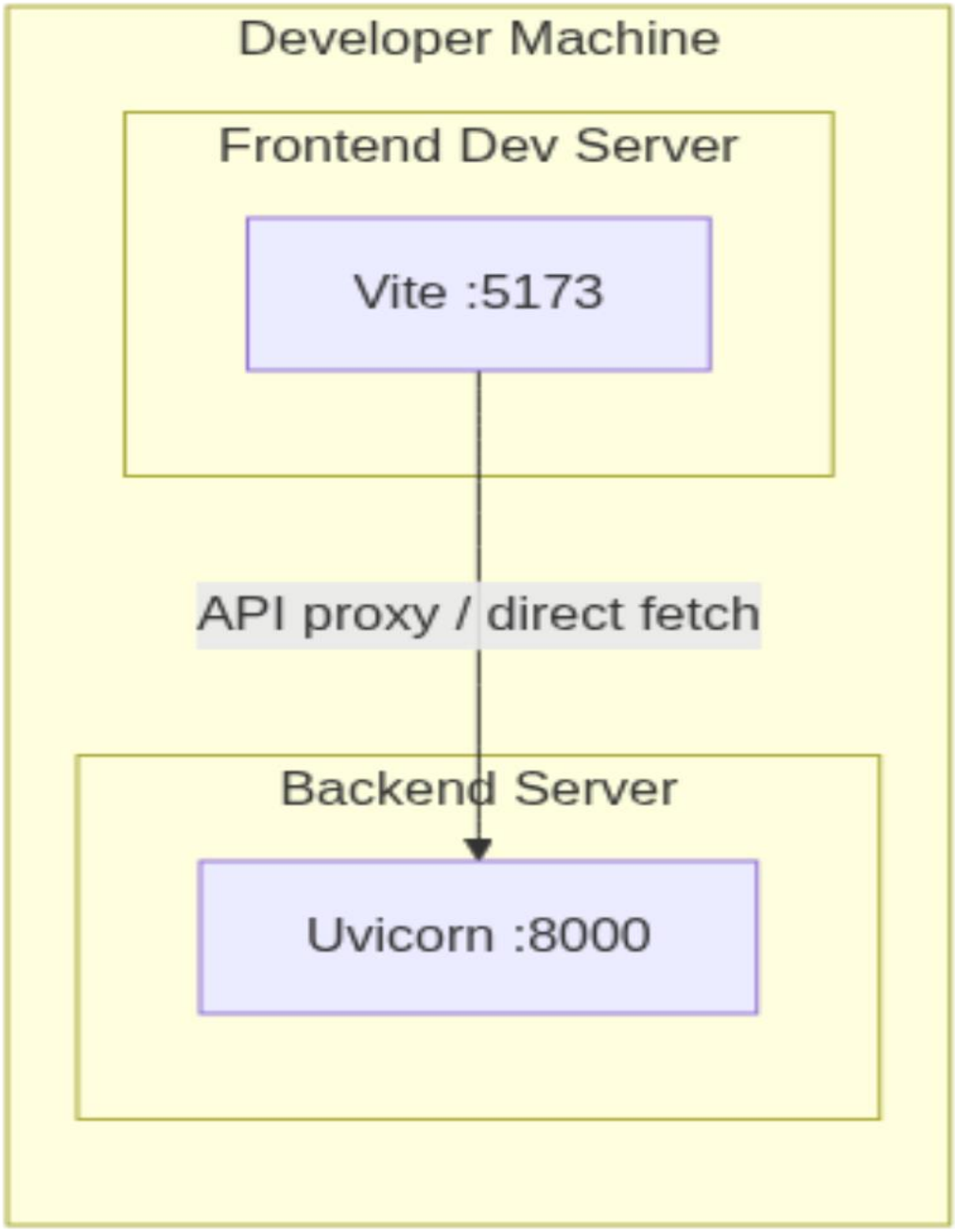
`/contact`	Contact	Contact form / information	No
`/login`	Log In	Email + password login form	No
`/signup`	Sign Up	Registration form (name, email, password)	No
`/admin-panel`	Admin Panel	View & restore soft-deleted car listings (Trash Bin)	Admin

Navigation Structure

Element	Links
Navbar (always visible)	Home, Cars, Brands, AI Pricing, About, Contact
Navbar (logged out)	Log In, Sign Up
Navbar (logged in)	User Name, Sell Your Car, Logout
Navbar (admin)	Admin badge, Trash Bin link
Footer	All main links + social links

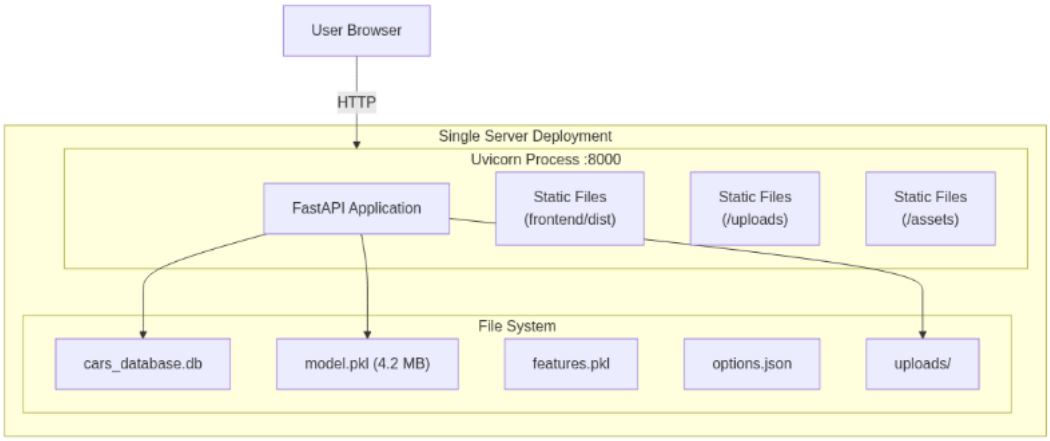
5.8 Deployment Architecture

Development Environment



Component	Command	Port
Frontend (dev)	`npm run dev`	`localhost:5173`
Backend (dev)	`python -m uvicorn main:app --reload`	`localhost:8000`

Production Deployment



Build & Deploy Steps

```
# 1. Build frontend
cd frontend
npm install
npm run build       # → generates frontend/dist/


# 2. Install backend dependencies
cd ../backend
pip install -r requirements.txt


# 3. (Optional) Train model
python train.py     # → generates model.pkl, features.pkl


# 4. (Optional) Create admin user
python create_admin.py


# 5. (Optional) Seed sample data
python import_cars.py


# 6. Start production server
python -m uvicorn main:app --host 0.0.0.0 --port 8000
```

Prerequisites

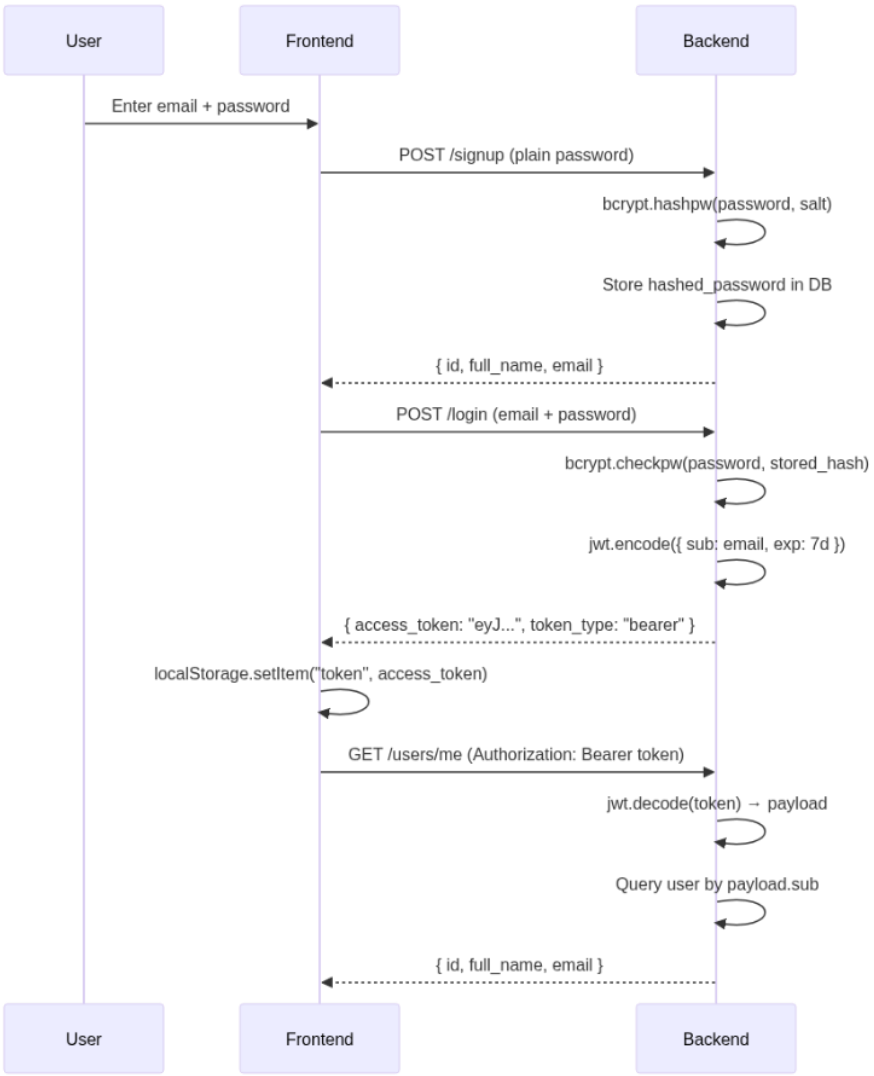
Requirement	Minimum Version
Python	3.8+
Node.js	18+
pip	Latest
npm	Comes with Node.js

5.9 Security Considerations

Authentication & Authorization

Mechanism	Implementation
Password Hashing	**bcrypt** with random salt (`bcrypt.gensalt()`)
Token Type	**JWT** (JSON Web Token)
Token Algorithm	HS256 (HMAC-SHA256)
Token Expiry	7 days (`60 * 24 * 7` minutes)
Token Storage	`localStorage` on the client
Admin Check	Hardcoded email comparison (admin@tara.com)

Authentication Flow



Security Layers

Layer	Measure	Details

Password Storage	bcrypt hashing	Passwords never stored in plaintext; salted with `bcrypt.gensalt()`
API Authentication	JWT Bearer tokens	Stateless, token contains user email as `sub` claim
Admin Authorization	Role-based (email check)	Admin endpoints check `payload.sub == "admin@tara.com"`
Soft Delete	`is_deleted` flag	Cars are never permanently deleted; admin can restore
CORS	Configured in middleware	Currently allows all origins (`*`) for development
Input Validation	Pydantic schemas	All API inputs validated with type-safe Pydantic models
File Uploads	UUID renaming	Uploaded images renamed with `uuid4().hex` to prevent path traversal
Error Handling	HTTPException	Standardized error responses with status codes and messages

Token Lifecycle

Event	Action
Signup	Auto-login after successful registration
Login	JWT token stored in `localStorage`
Page Load	Token validated via `GET /users/me`
Invalid Token	Token removed from `localStorage`, user logged out
Logout	Token removed from `localStorage`, state cleared
Expiry	Token expires after 7 days, next API call returns 401

Chapter 6 : Implementation & Testing

6.1 Technologies Used

The implementation phase of the project was carried out using a combination of modern data science, machine learning, and data visualization technologies. These

tools were selected based on their efficiency, scalability, and strong performance in handling structured automotive datasets.

The following technologies and libraries were used throughout the project:

Technology / Library	Purpose
Python	Main programming language used for data analysis and machine learning
Pandas	Data manipulation and preprocessing
NumPy	Numerical computations and mathematical operations
Matplotlib	Data visualization and plotting
Seaborn	Statistical data visualization and exploratory analysis
Scikit-learn	Machine learning utilities, preprocessing, evaluation metrics, and model training
XGBoost	Gradient boosting regression model
LightGBM	High-performance boosting framework
CatBoost	Gradient boosting model optimized for categorical features
RandomizedSearchCV	Hyperparameter tuning and model optimization
Jupyter Notebook	Development and experimentation environment

The implementation environment allowed efficient experimentation, preprocessing, visualization, model comparison, and performance evaluation throughout the development lifecycle.

6.2 Python Implementation

Python was selected as the primary programming language because of its flexibility, simplicity, and strong ecosystem for machine learning and data analysis applications.

The project implementation started by importing the required libraries responsible for:

- Data processing
- Visualization
- Model development
- Performance evaluation
- Hyperparameter optimization

The following libraries were imported during the implementation phase:

Import libraries

```
import numpy as np
import pandas as pd

import matplotlib.pyplot as plt
import seaborn as sns

from datetime import datetime

from sklearn.model_selection import train_test_split, RandomizedSearchCV
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.svm import SVR

from xgboost import XGBRegressor
from lightgbm import LGBMRegressor
from catboost import CatBoostRegressor

import warnings
warnings.filterwarnings('ignore')
```

The dataset was then loaded into the notebook environment using Pandas:

Load data

```
df = pd.read_csv(r"C:\Users\Lenovo\Desktop\Car Price Prediction\Car Price Prediction\Cars_Data.csv")
df.shape
```

```
(77119, 11)
```

After loading the dataset, initial exploration was performed using:

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 77119 entries, 0 to 77118
Data columns (total 11 columns):
 #   Column                     Non-Null Count  Dtype  
---  ------                     --------------  -----  
 0   price                      77119 non-null  int64  
 1   brand                      77119 non-null  object 
 2   model                      77119 non-null  object 
 3   color                      77119 non-null  object 
 4   registration_date          77119 non-null  object 
 5   power_kw                   77119 non-null  int64  
 6   power_ps                   77119 non-null  int64  
 7   transmission_type          77119 non-null  object 
 8   fuel_type                  77119 non-null  object 
 9   fuel_consumption_l_100km.1  77119 non-null  float64
 10  mileage_in_km              77119 non-null  int64  
dtypes: float64(1), int64(4), object(6)
memory usage: 6.5+ MB
```

This step helped identify:

- Data types
- Feature structure
- Missing values
- Numerical and categorical variables

Python provided a complete development environment for implementing all preprocessing, feature engineering, visualization, and machine learning tasks efficiently.

6.3 ML Implementation

The machine learning implementation phase focused on developing and evaluating multiple regression models capable of predicting vehicle prices accurately.

Several traditional and advanced machine learning algorithms were implemented to compare performance across different learning approaches.

The implemented models included:

- Linear Regression
- Decision Tree Regressor
- Support Vector Regressor (SVR)
- XGBoost Regressor
- LightGBM Regressor
- CatBoost Regressor

The models were selected because they represent:

- Linear learning techniques
- Tree-based learning methods
- Kernel-based regression
- Ensemble boosting algorithms

The implementation process included:

1. Data preprocessing
2. Feature engineering
3. Train-test splitting
4. Feature encoding
5. Model training
6. Performance evaluation
7. Hyperparameter tuning
8. Final model comparison

Advanced boosting algorithms demonstrated significantly stronger performance compared to traditional regression models due to their ability to capture nonlinear relationships within automotive pricing data.

6.4 Data Preprocessing Implementation

Data preprocessing represented one of the most critical phases of the project because machine learning performance strongly depends on data quality and consistency.

Several preprocessing operations were applied to prepare the dataset for training.

6.4.1 Duplicate Removal

Duplicate records were identified using:

```
df.duplicated().sum()
```

```
2674
```

After detection, duplicates were removed to improve data consistency:

```
]: df = df.drop_duplicates()
   df.shape
```

```
]: (74445, 11)
```

Removing duplicates helped prevent biased learning and repeated observations.

6.4.2 Cardinality Checking

Feature uniqueness and categorical diversity were analyzed using:

```
: for col in df.columns:
      print(f"{col}: {df[col].nunique()}")
```

```
price: 10206
brand: 29
model: 510
color: 14
registration_date: 331
power_kw: 400
power_ps: 400
transmission_type: 4
fuel_type: 4
fuel_consumption_l_100km.1: 312
mileage_in_km: 28919
```

This step helped understand:

- Category distribution
- Feature complexity

- Encoding requirements

6.4.3 Date Processing and Vehicle Age Extraction

The registration date feature was transformed into vehicle age to improve prediction relevance.

```
df["registration_date"] = pd.to_datetime(
    df["registration_date"],
    format="%d-%m-%y",
    errors="coerce"
)

current_year = datetime.now().year
df["vehicle_age"] = current_year - df["registration_date"].dt.year

df.drop(columns=["registration_date"], inplace=True)
```

```
df["vehicle_age"].describe()
```

Vehicle age provides more meaningful predictive information than raw registration dates.

6.4.4 Logical Data Cleaning

Logical filtering was applied to remove unrealistic records:

```
df = df[df["mileage_in_km"] > 0]
df = df[df["vehicle_age"] >= 0]
df.shape
```

```
(74364, 11)
```

This ensured that:

- Mileage values remained valid
- Vehicle age remained logically consistent

6.4.5 Target Distribution Analysis

The price distribution was analyzed using histogram visualization:

```
plt.figure(figsize=(8,4))
sns.histplot(df["price"], bins=50, kde=True)
plt.title("Price Distribution")
plt.show()
```

The visualization revealed strong right-skewness caused by luxury and high-performance vehicles.

6.4.6 Outlier Handling Using Winsorization

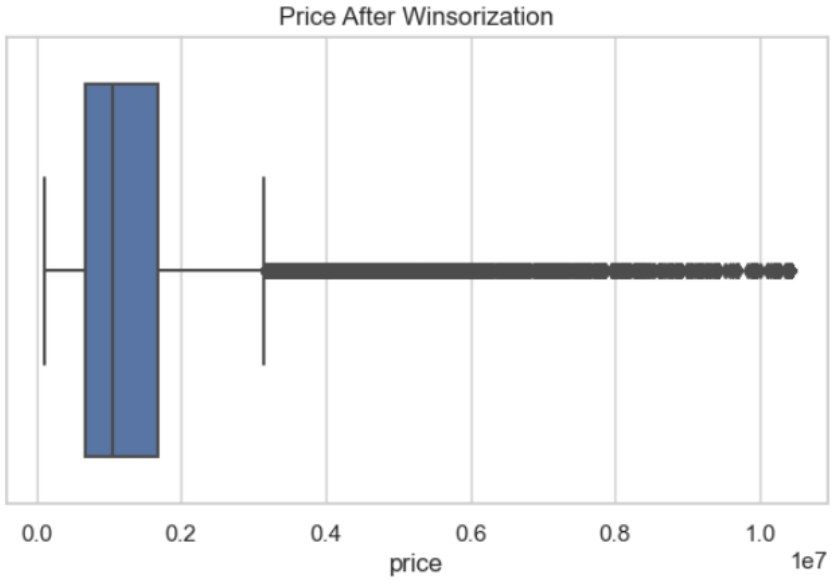
Instead of removing outliers completely, Winsorization was applied to reduce extreme value influence while preserving dataset size.

The updated distribution was visualized using:

```
p_low, p_high = df["price"].quantile([0.01, 0.99])
df["price"] = df["price"].clip(p_low, p_high)
```

```
p_low, p_high = df["price"].quantile([0.01, 0.99])
df["price"] = df["price"].clip(p_low, p_high)
```

```
plt.figure(figsize=(7,4))
sns.boxplot(x=df["price"])
plt.title("Price After Winsorization")
plt.show()
```



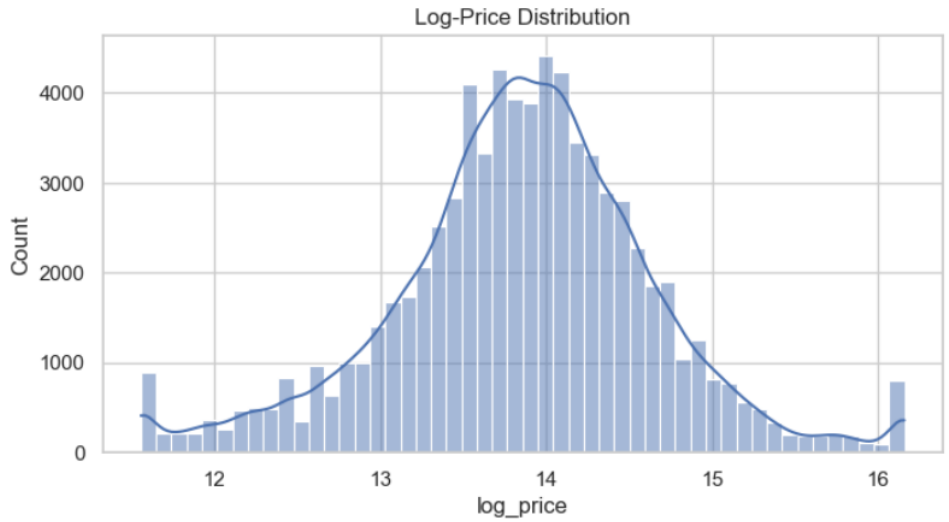
This approach improved model stability without losing valuable luxury car information.

6.4.7 Log Transformation

To normalize the target distribution and reduce skewness, logarithmic transformation was applied:

```
df["log_price"] = np.log1p(df["price"])
```

```
plt.figure(figsize=(8,4))
sns.histplot(df["log_price"], bins=50, kde=True)
plt.title("Log-Price Distribution")
plt.show()
```



This transformation significantly improved regression model performance.

6.4.8 Redundancy Analysis

Correlation analysis was performed between power-related features:

```
df[["power_kw", "power_ps"]].corr()
```

	power_kw	power_ps
power_kw	1.000000	0.999996
power_ps	0.999996	1.000000

This step helped identify feature redundancy and relationships between horsepower measurements.

6.4.9 Handling Unknown Transmission Values

A binary indicator was created for unknown transmission types:

Power features (remove redundancy)

```
df["transmission_unknown_flag"] = (df["transmission_type"] == "Unknown").astype(int)
```

This preserved potentially useful information instead of removing incomplete records.

6.5 Feature Engineering Implementation

Feature engineering was applied to create additional meaningful variables capable of improving prediction accuracy and capturing real-world vehicle behavior.

6.5.1 Mileage Per Year Feature

A new feature called `km_per_year` was created to measure average yearly vehicle usage.

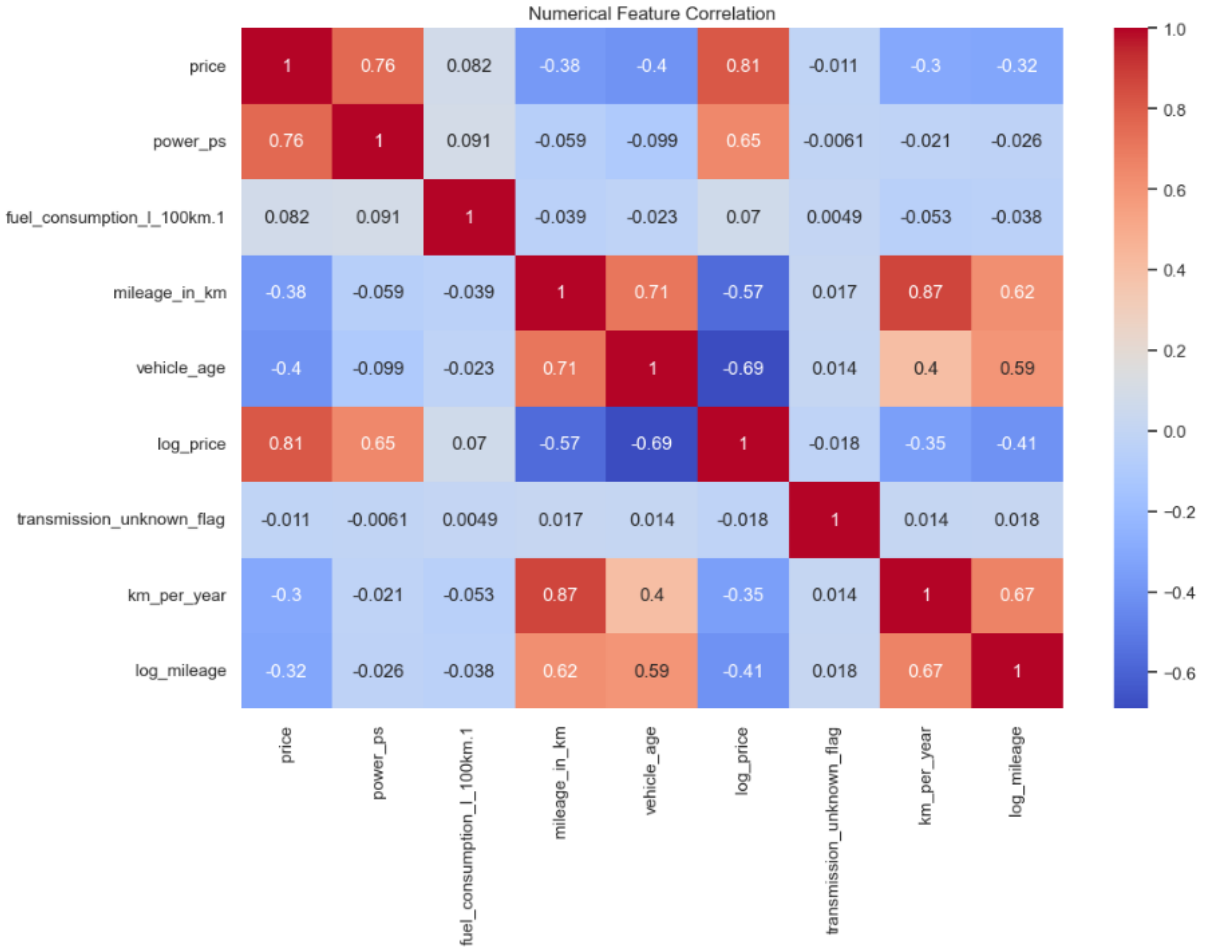
```
df["km_per_year"] = df["mileage_in_km"] / (df["vehicle_age"] + 1)
df["log_mileage"] = np.log1p(df["mileage_in_km"])
```

6.5.3 Correlation Heatmap Analysis

Numerical feature relationships were analyzed using a correlation heatmap.

```
num_cols = df.select_dtypes(include=np.number).columns

plt.figure(figsize=(12,8))
sns.heatmap(df[num_cols].corr(),annot=True , cmap="coolwarm")
plt.title("Numerical Feature Correlation")
plt.show()
```



The heatmap helped identify:

- Strong correlations
- Redundant variables
- Important predictive features

6.5.4 Categorical Feature Exploration

Categorical feature distributions were explored visually:

```
cat_cols = df.select_dtypes(include="object").columns

for col in cat_cols:
    plt.figure(figsize=(6,3))
    df[col].value_counts().head(10).plot(kind="bar")
    plt.title(f"Top 10 {col}")
    plt.show()
```

This analysis helped understand:

- Brand dominance
- Popular vehicle models
- Fuel and transmission distributions

6.6 Model Training Process

The machine learning pipeline started by defining features and prediction targets.

6.6.1 Feature and Target Definition

```
target = "log_price"
X = df.drop(columns=["price", "log_price"])
y = df[target]
```

```
cat_cols = [
    "brand",
    "model",
    "color",
    "transmission_type",
    "fuel_type"
]

num_cols = [c for c in X.columns if c not in cat_cols]
```

6.6.2 Train-Test Split

The dataset was divided into training and testing sets:

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

An 80/20 split was used to ensure proper generalization evaluation.

6.6.3 One-Hot Encoding

For Scikit-learn models, categorical variables were encoded using One-Hot Encoding:

```
from sklearn.preprocessing import OneHotEncoder

encoder = OneHotEncoder(
    handle_unknown="ignore",
    sparse=False
)

X_train_cat_enc = encoder.fit_transform(X_train[cat_cols])
X_test_cat_enc  = encoder.transform(X_test[cat_cols])

X_train_sklearn = np.hstack([X_train[num_cols].values, X_train_cat_enc])
X_test_sklearn  = np.hstack([X_test[num_cols].values, X_test_cat_enc])
```

Encoded datasets were then combined with numerical features.

6.6.4 Model Definition

```
models = {
    "LinearRegression": LinearRegression(),
    "DecisionTreeRegressor": DecisionTreeRegressor(max_depth=5, random_state=42),
    "SVR": SVR(kernel="rbf"),
    "XGBoost": XGBRegressor(
        n_estimators=200,
        learning_rate=0.1,
        max_depth=6,
        random_state=42,
        n_jobs=-1
    ),
    "LightGBM": LGBMRegressor(random_state=42),
    "CatBoost": CatBoostRegressor(
        silent=True,
        random_seed=42
    )
}
```

6.6.5 Model Evaluation

Models were evaluated using:

- MAE
- RMSE
- R^2 Score

The comparison results showed:

Results comparison

```
]: results_df = pd.DataFrame(results).sort_values("R2", ascending=False)
   results_df
```

]:

	Model	MAE	RMSE	R2
5	CatBoost	1.676757e+05	3.438268e+05	9.397368e-01
3	XGBoost	1.776860e+05	3.613936e+05	9.334216e-01
4	LightGBM	1.814433e+05	3.702106e+05	9.301333e-01
1	DecisionTreeRegressor	3.411223e+05	6.392122e+05	7.917128e-01
2	SVR	5.936598e+05	1.237915e+06	2.188145e-01
0	LinearRegression	1.015344e+94	1.200707e+96	-7.349313e+179

CatBoost achieved the highest predictive performance before tuning.

6.7 Hyperparameter Tuning

To further improve model performance, hyperparameter optimization was applied using RandomizedSearchCV.

This technique searches for optimal parameter combinations efficiently while reducing computational cost.

6.7.1 XGBoost Tuning

The following parameters were optimized:

- n_estimators
- max_depth
- learning_rate
- subsample
- colsample_bytree

RandomizedSearchCV for XGBoost

```
xgb_param_grid = {
    "n_estimators": [300, 500, 800],
    "max_depth": [4, 6, 8],
    "learning_rate": [0.01, 0.05, 0.1],
    "subsample": [0.7, 0.9, 1.0],
    "colsample_bytree": [0.7, 0.9, 1.0]
}

xgb_model = XGBRegressor(
    random_state=42,
    n_jobs=-1
)

xgb_search = RandomizedSearchCV(
    estimator=xgb_model,
    param_distributions=xgb_param_grid,
    n_iter=20,
    scoring="neg_mean_absolute_error",
    cv=3,
    random_state=42,
    verbose=1,
    n_jobs=-1
)

print("Tuning XGBoost...")
xgb_search.fit(X_train_sklearn, y_train)

best_xgb = xgb_search.best_estimator_
print("Best XGBoost Params:", xgb_search.best_params_)
```

```
Tuning XGBoost...
Fitting 3 folds for each of 20 candidates, totalling 60 fits
Best XGBoost Params: {'subsample': 0.7, 'n_estimators': 800, 'max_depth': 6, 'learning_rate': 0.1, 'colsample_bytree': 0.9}
```

6.7.2 LightGBM Tuning

The following parameters were tuned:

- num_leaves
- learning_rate
- max_depth
- subsample
- n_estimators

This improved generalization performance and reduced prediction error.

RandomizedSearchCV for LightGBM

```
lgb_param_grid = {
    "n_estimators": [300, 500, 800],
    "learning_rate": [0.01, 0.05, 0.1],
    "num_leaves": [31, 63, 127],
    "max_depth": [-1, 6, 10],
    "subsample": [0.7, 0.9, 1.0]
}

lgb_model = LGBMRegressor(
    random_state=42
)

lgb_search = RandomizedSearchCV(
    estimator=lgb_model,
    param_distributions=lgb_param_grid,
    n_iter=20,
    scoring="neg_mean_absolute_error",
    cv=3,
    random_state=42,
    verbose=1,
    n_jobs=-1
)

print("Tuning LightGBM...")
lgb_search.fit(X_train_sklearn, y_train)

best_lgb = lgb_search.best_estimator_
print("Best LightGBM Params:", lgb_search.best_params_)
```

6.7.3 CatBoost Tuning

CatBoost tuning included:

- iterations
- depth
- learning_rate
- l2_leaf_reg
- bagging_temperature
- random_strength

CatBoost showed the strongest final performance after optimization.

RandomizedSearchCV for CatBoost

```
cat_param_grid = {
    "iterations": [1000, 1500, 2000, 3000],
    "depth": [6, 8, 10],
    "learning_rate": [0.01, 0.03, 0.05],
    "l2_leaf_reg": [3, 5, 7],
    "bagging_temperature": [0, 1, 5],
    "random_strength": [5, 10]
}

cat_model = CatBoostRegressor(
    loss_function="RMSE",
    eval_metric="R2",
    random_seed=42,
    silent=True
)

cat_search = RandomizedSearchCV(
    estimator=cat_model,
    param_distributions=cat_param_grid,
    n_iter=20,
    scoring="neg_mean_absolute_error",
    cv=3,
    random_state=42,
    verbose=1,
    n_jobs=-1
)

print("Tuning CatBoost...")
cat_search.fit(
    X_train_cat,
    y_train,
    cat_features=cat_cols
)

best_cat = cat_search.best_estimator_
print("Best CatBoost Params:", cat_search.best_params_)
```

6.7.4 Final Tuned Model Results

	Model	MAE	RMSE	R2
2	CatBoost	160115.546783	334327.859950	0.943021
1	LightGBM	160140.879463	336024.426852	0.942441
0	XGBoost	161459.610276	340006.150886	0.941069

The tuned CatBoost model achieved the highest prediction accuracy and was selected as the final deployment model.

6.7.5 Train and Test Performance Comparison

	Model	Train_R2	Test_R2
0	CatBoost	0.953196	0.943021
1	LightGBM	0.953414	0.942441
2	XGBoost	0.957874	0.941069

The close relationship between training and testing scores indicates that the models generalized effectively without significant overfitting.

Overall, the implementation and testing phase demonstrated the effectiveness of ensemble boosting algorithms in predicting real-world automotive prices with high accuracy and strong generalization capability.

6.8 Model Evaluation

Model evaluation was conducted to measure the predictive performance and reliability of the implemented machine learning algorithms. Multiple regression models were trained and compared using several evaluation metrics to determine the most accurate model for vehicle price prediction.

The evaluation process focused on:

- Prediction accuracy
- Error minimization
- Generalization capability
- Model stability

The following machine learning models were evaluated:

- Linear Regression
- Decision Tree Regressor
- Support Vector Regressor (SVR)
- XGBoost Regressor
- LightGBM Regressor
- CatBoost Regressor

After both training and hyperparameter optimization, the ensemble boosting models achieved significantly stronger results compared to traditional machine learning methods.Among all tested models, CatBoost achieved the highest overall performance and was selected as the final prediction model for deployment.

6.8.1 Initial Model Comparison

The initial evaluation results before hyperparameter tuning are presented below:

Model	MAE	RMSE	R^2 Score
CatBoost	167,675.69	343,826.82	0.9397
XGBoost	177,686.00	361,393.60	0.9334
LightGBM	181,443.30	370,210.60	0.9301
Decision Tree Regressor	341,122.30	639,212.20	0.7917
SVR	593,659.80	1,237,915.00	0.2188
Linear Regression	Extremely Poor	Extremely Poor	Negative Value

6.8.2 Final Tuned Model Evaluation

After applying RandomizedSearchCV for hyperparameter optimization, model performance improved further.

Model	MAE	RMSE	R^2 Score
CatBoost	160,115.55	334,327.86	0.943021
LightGBM	160,140.88	336,024.43	0.942441
XGBoost	161,459.61	340,006.15	0.941069

The tuned CatBoost model achieved the best overall predictive performance and demonstrated strong generalization capability on unseen data.

6.9 Testing Process

The testing process was designed to evaluate the robustness, reliability, and predictive consistency of the proposed AI system.

Several testing stages were conducted throughout the implementation phase.

6.9.1 Data Validation Testing

The dataset was tested to ensure:

- Correct feature types
- Valid mileage values
- Logical vehicle age
- Proper categorical formatting
- Clean numerical ranges

Duplicate records and invalid observations were removed before training.

6.9.2 Model Training Testing

Each machine learning model was tested independently using the same training and testing datasets to ensure fair comparison.

The dataset was divided using an 80/20 train-test split:

- 80% for training
- 20% for testing

This testing strategy helped evaluate how well the models generalized to unseen vehicle data.

6.9.3 Prediction Accuracy Testing

The generated predictions were compared against actual vehicle prices using regression evaluation metrics.

Testing focused on:

- Prediction precision
- Error reduction

- Stability across multiple vehicle categories

6.9.4 Overfitting Testing

To ensure the models were not memorizing the training data, Train R^2 and Test R^2 scores were compared.

Model	Train R^2	Test R^2
CatBoost	0.953196	0.943021
LightGBM	0.953414	0.942441
XGBoost	0.957874	0.941069

The close relationship between training and testing performance indicates that the models generalized effectively without severe overfitting.

6.9.5 Visualization Testing

Prediction quality was visually evaluated using:

- Correlation heatmaps
- Distribution plots
- Boxplots
- Actual vs Predicted scatter plots

These visualizations helped verify prediction consistency and data quality.

6.10 Performance Metrics

Several regression evaluation metrics were used to assess the performance of the machine learning models.

6.10.1 Mean Absolute Error (MAE)

MAE measures the average absolute difference between actual prices and predicted prices.

Lower MAE values indicate higher prediction accuracy.

```
maen = mean_absolute_error(y_test_real, y_pred_real)
```

6.10.2 Mean Squared Error (MSE)

MSE measures the average squared prediction error.

This metric penalizes large prediction errors more heavily.

```
mse = mean_squared_error(y_test_real, y_pred_real)
```

6.10.3 Root Mean Squared Error (RMSE)

RMSE represents the square root of the Mean Squared Error and provides interpretable error measurements in the original price scale.

```
rmse = np.sqrt(mse)
```

6.10.4 R^2 Score

R^2 measures how well the model explains variance in vehicle prices.

An R^2 value closer to 1 indicates stronger predictive performance.

```
r2 = r2_score(y_test_real, y_pred_real)
```

6.10.5 Final CatBoost Performance

The final deployed CatBoost model achieved:

Metric	Value
MAE	160,115.55
RMSE	334,327.86
R^2 Score	0.943021

These results confirm the strong predictive capability of the proposed AI system.

6.12 Results Analysis

The experimental results demonstrate that ensemble boosting algorithms significantly outperform traditional regression techniques in vehicle price prediction tasks.

The study revealed several important findings:

- Advanced boosting models achieved the highest prediction accuracy.
- CatBoost demonstrated superior handling of categorical vehicle features.

- Log transformation and feature engineering significantly improved model performance.
- Winsorization effectively reduced the influence of extreme outliers without removing important luxury vehicle observations.
- Vehicle age and mileage-related features showed strong influence on pricing behavior.
- Ensemble learning methods captured nonlinear relationships more effectively than linear models.

The final CatBoost model achieved an R^2 score of 0.943021, indicating that the model successfully explained approximately 94% of the variance in vehicle prices.

Chapter 7 : Conclusion & Future Work

7.1 Project Conclusion

The Car Price Prediction System was successfully developed to address one of the most common challenges in the used-car market: accurately estimating the value of a vehicle based on its characteristics. By leveraging machine learning techniques and real-world automotive data, the project provides a reliable and data-driven solution for car price estimation.

Throughout the project lifecycle, a complete end-to-end machine learning pipeline was implemented, starting from data collection and preprocessing, followed by feature engineering, exploratory data analysis, model training, hyperparameter optimization, and final deployment planning.

Several machine learning algorithms were evaluated and compared, including Linear Regression, Decision Tree Regressor, Support Vector Regressor (SVR), XGBoost, LightGBM, and CatBoost. Experimental results demonstrated that advanced boosting algorithms significantly outperformed traditional regression approaches.

The final system successfully combines business understanding, data science methodologies, and modern software engineering practices into a practical AI-powered solution capable of supporting real-world decision-making in the automotive industry.

7.2 Final Findings

The experimental analysis produced several important findings regarding car price prediction and machine learning model performance.

Key Findings

- Vehicle age is one of the most influential factors affecting used-car prices.
- Mileage has a strong negative correlation with vehicle value.
- Brand and model information significantly impact price estimation accuracy.
- Feature engineering substantially improved model performance.
- Logarithmic transformation of the target variable improved prediction stability.
- Ensemble learning models consistently outperformed traditional machine learning techniques.

Final Model Performance

Model	R^2 Score
CatBoost	0.943021
LightGBM	0.942441
XGBoost	0.941069

Among all evaluated models, CatBoost achieved the highest prediction accuracy and was selected as the final production model.

These findings confirm that gradient boosting algorithms are highly effective for solving complex regression problems involving structured tabular datasets.

7.3 Business Impact

The developed system has the potential to provide significant value to multiple stakeholders within the automotive market.

Benefits for Car Buyers

- Provides fair market price estimation.
- Reduces the risk of overpaying.
- Supports informed purchasing decisions.

Benefits for Car Sellers

- Assists in setting competitive and realistic prices.
- Reduces pricing uncertainty.
- Improves sales efficiency.

Benefits for Dealerships

- Accelerates vehicle valuation processes.

- Reduces dependency on manual assessments.
- Supports inventory management and pricing strategies.

Benefits for Online Automotive Platforms

- Enhances user experience through instant price estimation.
- Increases platform credibility and trust.
- Supports intelligent recommendation systems.

The project demonstrates how artificial intelligence can transform traditional pricing processes into automated and data-driven decision systems.

7.4 Technical Achievements

Several technical milestones were successfully accomplished during the development of this project.

Data Engineering Achievements

- Data cleaning and preprocessing pipeline implemented.
- Duplicate and invalid records removed.
- Outlier treatment performed using Winsorization.
- Date features transformed into meaningful numerical variables.

Feature Engineering Achievements

- Created Vehicle Age feature.
- Created Kilometers Per Year feature.
- Applied Log Mileage transformation.
- Added Transmission Unknown Indicator feature.
- Reduced feature redundancy through correlation analysis.

Machine Learning Achievements

- Implemented six regression algorithms.
- Performed model comparison and benchmarking.

- Applied hyperparameter tuning using RandomizedSearchCV.
- Evaluated models using multiple performance metrics.
- Selected the best-performing model for deployment.

Software Engineering Achievements

- Designed a complete AI-powered web system architecture.
- Developed frontend-backend communication workflow.
- Designed scalable deployment architecture.
- Produced UML diagrams and system documentation.

These achievements demonstrate the integration of machine learning engineering, software development, and business analytics within a single project.

7.5 Project Limitations

Although the project achieved strong results, several limitations should be acknowledged.

Dataset Limitations

- The dataset size is relatively small compared to commercial automotive datasets.
- The data primarily represents specific vehicle markets and may not fully generalize to all regions.
- Some potentially useful vehicle attributes were unavailable.

Model Limitations

- Predictions are dependent on the quality of input data.
- Unexpected market fluctuations cannot be fully captured by historical data.
- The model does not currently incorporate external economic indicators.

System Limitations

- Real-time market data integration is not yet available.
- Continuous automated retraining has not been implemented.
- User feedback mechanisms are not included in the current version.

These limitations provide opportunities for future development and enhancement.

7.6 Future Improvements

Several improvements can further enhance the system's accuracy, scalability, and business value.

Data Improvements

- Collect larger datasets containing millions of vehicle records.
- Integrate data from multiple automotive marketplaces.
- Include vehicle maintenance and service history information.
- Incorporate accident and ownership records.

Model Improvements

- Explore advanced deep learning architectures.
- Investigate stacking and blending ensemble methods.
- Implement automated feature selection techniques.
- Develop explainable AI capabilities for prediction interpretation.

System Improvements

- Deploy the system on cloud infrastructure.
- Add real-time model monitoring.
- Implement continuous model retraining pipelines.
- Build a mobile application version.
- Enable multilingual support.

These improvements can increase both technical performance and user adoption.

7.7 AI Future Opportunities

Artificial Intelligence offers significant opportunities for extending this project beyond simple price prediction.

Intelligent Vehicle Recommendation Systems

Future versions could recommend vehicles based on:

- Budget
- Fuel efficiency
- Performance requirements
- Family size
- User preferences

Automated Market Analysis

AI models could continuously monitor market trends and provide:

- Demand forecasting
- Price trend prediction
- Vehicle depreciation analysis

Personalized Pricing Systems

Machine learning could generate customized pricing strategies for dealerships and individual sellers.

Computer Vision Integration

Future systems could analyze vehicle images and automatically estimate:

- Vehicle condition
- Exterior damage
- Paint quality
- Visual depreciation factors

Generative AI Assistants

AI assistants could guide users through the buying and selling process by providing:

- Vehicle recommendations
- Price negotiation support
- Market insights
- Financing suggestions

Smart Automotive Ecosystems

The project can serve as a foundation for larger intelligent automotive platforms that combine machine learning, predictive analytics, recommendation systems, and real-time market intelligence.

Chapter Summary

This project successfully demonstrated the application of modern machine learning techniques to the problem of used-car price prediction. Through comprehensive data preprocessing, feature engineering, model comparison, and hyperparameter optimization, the developed system achieved a high prediction accuracy of **94.30% (R^2 = 0.943021)** using the CatBoost algorithm.

The results validate the effectiveness of ensemble learning methods for structured automotive data and highlight the growing role of artificial intelligence in supporting business decisions. While several limitations remain, the project establishes a strong foundation for future research, system expansion, and real-world deployment within the automotive industry.